



Lexical analysis, Parsing, Syntax-directed translation, Runtime environments, Intermediate code generation.

Mark Distribution in Previous GATE

Year	2026 - 1	2026 - 2	2025 - 1	2025 - 2	2024 - 1	2024 - 2	2023	2022	2021 - 1	2021 - 2	Minimum
1 Mark Count	2	2	2	2	2	2	1	2	1	2	1
2 Marks Count	2	2	2	2	4	3	3	1	3	2	1
Total Marks	6	6	6	6	10	8	7	4	7	6	4

Subject Overview

Compiler Design is a fundamental area of Computer Science that deals with the theory and practice of building compilers – programs that translate source code written in a high-level programming language into an equivalent low-level machine-understandable form (like assembly or machine code). Understanding compiler design is crucial for a GATE CS aspirant as it provides deep insights into how programming languages work, how software interacts with hardware, and the underlying mechanisms of program execution. This subject often carries a weightage of 6-10 marks in the GATE CS exam, typically involving 2-3 questions of 2 marks and 1-2 questions of 1 mark. Questions range from theoretical concepts (phases of compilation, definitions) and conceptual understanding (optimization techniques, runtime environment) to numerical problems (parsing table construction, FIRST/FOLLOW sets, DAG generation, register allocation). A strong grasp of this subject is vital not just for the exam, but also for a holistic understanding of computer systems.

Topic-wise Key Concepts

Abstract Syntax Tree (AST)

An AST is a tree representation of the abstract syntactic structure of source code, where each node denotes a construct in the source code. It abstracts away the concrete syntax (like parentheses or semicolons) and focuses on the essential structure and meaning of the program. ASTs are crucial for semantic analysis, intermediate code generation, and various optimization techniques.

- **Key Properties:**

- Nodes represent operators and keywords, while leaves represent operands and identifiers.
- More compact than a parse tree, omitting unnecessary details like non-terminals that only derive other non-terminals.
- Used as an intermediate representation for subsequent compiler phases.

- **Common Pitfalls:** Confusing AST with a parse tree. A parse tree shows every detail of the derivation; an AST shows only the essential structure.

- **Standard Problem-Solving Technique:** Given a grammar and an expression, construct its parse tree first, then derive the AST by eliminating redundant nodes and structuring it based on operator precedence and associativity.

Ambiguous Grammar

A grammar is said to be ambiguous if there exists at least one string that can be generated by the grammar in more than one leftmost derivation, or more than one rightmost derivation, or has more than one parse tree. Ambiguity is undesirable in programming language grammars because it leads to multiple interpretations of the same program statement.

- **Key Properties:**

- Causes difficulty for parsers as they cannot uniquely determine the structure of an input string.
- Common sources include dangling-else problem and lack of operator precedence/associativity rules.

- **Common Pitfalls:** Not being able to identify ambiguity. A common test is to find a string with two distinct parse trees.

- **Standard Problem-Solving Technique:** To resolve ambiguity, rewrite the grammar by introducing new non-terminals or by incorporating precedence and associativity rules directly into the grammar productions. For example, for arithmetic expressions, introduce non-terminals like Term and Factor.

Assembler

An assembler is a program that translates assembly language code into machine code. Assembly language uses

mnemonics for operations and symbolic names for memory locations, making it more human-readable than raw machine code. Assemblers typically perform a one-to-one translation of assembly instructions to machine instructions.

- **Key Properties:**

- **Two-Pass Assembler:**

1. **Pass 1:** Scans the source code to build a symbol table (mapping symbolic labels to memory addresses) and determines the length of machine instructions.
2. **Pass 2:** Uses the symbol table to translate assembly instructions into machine code, filling in operand addresses.

- Handles pseudo-operations (directives) like `ORG`, `EQU`, `START`, `END`, which guide the assembly process but don't translate into machine instructions.

- **Common Pitfalls:** Confusing assembler with compiler or linker. Assembler works at a lower level, translating assembly to machine code.

- **Standard Problem-Solving Technique:** Trace the assembly process, building a symbol table and calculating location counter values for each instruction and data item.

Backpatching

Backpatching is a technique used in one-pass code generation, particularly for control flow statements (like if-else, while loops) and boolean expressions. It involves generating jump instructions with initially unspecified target addresses, which are later filled in (patched) once the true target address becomes known.

- **Key Properties:**

- Uses lists of incomplete jumps (e.g., `truelist`, `falselist`, `nextlist`).
 - `makelist(i)`: Creates a new list containing only `i` (an index into the list of quadruples).
 - `merge(p1, p2)`: Concatenates lists `p1` and `p2`, returning the combined list.
 - `backpatch(p, i)`: Inserts `i` as the target label for each of the jumps in `list p`.

- **Standard Problem-Solving Technique:** Trace the generation of three-address code for control flow statements, maintaining and updating `truelist`, `falselist`, and `nextlist` as new quadruples are generated.

Basic Blocks

A basic block is a sequence of consecutive statements in which flow of control enters at the beginning and leaves at the end without halt or possibility of branching except at the end. Basic blocks are fundamental units for many code optimization techniques.

- **Key Properties:**

- No jumps into the middle of a basic block.
 - No jumps out of the middle of a basic block.
 - The first statement is called a **leader**.

- **Standard Problem-Solving Technique:**

1. Identify **leaders**:
 - The first statement of the program is a leader.
 - Any statement that is the target of a conditional or unconditional jump is a leader.
 - Any statement immediately following a conditional or unconditional jump is a leader.
2. For each leader, its basic block consists of the leader and all subsequent statements up to, but not including, the next leader or the end of the program.

Code Optimization

Code optimization is the phase of a compiler that attempts to improve the intermediate code or target code to make it run faster, use less memory, or consume less power, without changing the program's observable behavior. It can be machine-independent or machine-dependent.

- **Key Types/Techniques:**

- **Machine-Independent:**

- **Peephole Optimization:** Examines a small sliding window of instructions and replaces suboptimal sequences with better ones (e.g., redundant loads/stores, strength reduction).
 - **Common Subexpression Elimination (CSE):** Identifies and removes redundant computations of the same expression.
 - **Dead Code Elimination:** Removes code that is never executed or whose results are never used.
 - **Constant Folding:** Evaluates constant expressions at compile time (e.g., $2 * 3 + 5$ becomes 11).

- **Loop Optimizations:** Loop invariant code motion, induction variable elimination, loop unrolling.
- **Strength Reduction:** Replacing expensive operations with cheaper ones (e.g., $x * 2$ with $x + x$ or $x << 1$).
- **Machine-Dependent:** Register allocation, instruction scheduling.
- **Key Property:** Optimization must preserve the semantic equivalence of the program.
- **Common Pitfalls:** Misidentifying opportunities for optimization or applying an optimization that changes program behavior.

Compilation Phases

The compilation process is typically divided into several phases, each performing a specific task in the translation of source code to target code. These phases operate sequentially, with the output of one phase serving as the input to the next.

- **Phases in Order:**
 1. **Lexical Analysis (Scanning):** Breaks source code into tokens.
 2. **Syntax Analysis (Parsing):** Checks grammar rules and builds a parse tree/AST.
 3. **Semantic Analysis:** Checks for type compatibility, undeclared variables, etc. (meaning of the program).
 4. **Intermediate Code Generation:** Creates an abstract machine-independent representation (e.g., three-address code).
 5. **Code Optimization:** Improves the intermediate or target code for efficiency.
 6. **Code Generation:** Translates intermediate code into target machine code (e.g., assembly).
- **Supporting Components:**
 - **Symbol Table Management:** Stores information about identifiers, used across multiple phases.
 - **Error Handling:** Detects and reports errors at each phase.
- **Common Pitfalls:** Confusing the order or specific responsibilities of each phase.

Compiler Tokenization (Lexical Analysis)

Compiler tokenization, or Lexical Analysis, is the first phase of a compiler where the input stream of characters is grouped into meaningful sequences called **tokens**. Each token represents a basic unit of the language, such as keywords, identifiers, operators, or literals.

- **Core Idea:** Converts a stream of characters into a stream of tokens.
- **Key Properties:**
 - Tokens are defined by **regular expressions**.
 - Lexical analyzers (scanners) are typically implemented using **Finite Automata** (NFA or DFA).
 - Handles whitespace and comments, usually discarding them.
 - **Longest Match Rule:** When multiple token patterns match a prefix of the input, the longest possible token is chosen.
 - **Precedence Rule:** If multiple patterns match the same longest prefix, the pattern listed first (or with higher precedence) is chosen.
- **Common Pitfalls:** Incorrectly applying longest match or precedence rules.
- **Standard Problem-Solving Technique:** Given a string and a set of regular expressions for tokens, identify the sequence of tokens.

Directed Acyclic Graph (DAG)

A DAG is a directed graph data structure used as an intermediate representation for basic blocks in compilers. Nodes in a DAG represent operations, and edges represent the flow of values between operations. It effectively highlights common subexpressions and the order of operations within a basic block.

- **Key Properties:**
 - Each node has a unique value. If an expression is computed multiple times, it corresponds to a single node in the DAG.
 - Leaf nodes are identifiers or constants. Interior nodes are operators.
 - Used for common subexpression elimination, dead code elimination, and reordering of operations.
- **Standard Problem-Solving Technique:**
 1. For each statement $x = y \text{ op } z$:
 - If y or z are not in the DAG, create nodes for them.
 - If op is not in the DAG with children y and z , create a node for it.
 - Add x as a label to the node representing $y \text{ op } z$.
 2. If $x = y$, make x a label for the node representing y .

Expression Evaluation

Expression evaluation is the process of computing the value of an arithmetic or logical expression according to operator precedence and associativity rules. Compilers often convert infix expressions to postfix or prefix notation for easier evaluation using a stack-based approach.

- **Key Notations:**

- **Infix:** Operators between operands (e.g., $a + b * c$).
- **Postfix (Reverse Polish Notation - RPN):** Operators after operands (e.g., $a \ b \ c \ * \ +$).
- **Prefix (Polish Notation):** Operators before operands (e.g., $+ \ a \ * \ b \ c$).

- **Key Properties:**

- **Precedence:** Determines which operator is evaluated first (e.g., multiplication before addition).
- **Associativity:** Determines evaluation order for operators of the same precedence (e.g., left-to-right for $+$, $-$, $*$, $/$; right-to-left for $^$).

- **Standard Problem-Solving Technique:**

- **Infix to Postfix/Prefix:** Use a stack to manage operators based on precedence and associativity.
- **Postfix Evaluation:** Scan from left to right. Push operands onto a stack. When an operator is encountered, pop required operands, perform operation, push result back.

First and Follow

FIRST and FOLLOW sets are crucial for constructing predictive (LL(1)) parsing tables and for resolving ambiguities in grammars. They help determine which production to apply or which token to expect next.

- **FIRST(X):** The set of terminal symbols that begin strings derived from X. If X can derive ϵ (empty string), then ϵ is in FIRST(X).
- **FOLLOW(A):** The set of terminal symbols that can immediately follow the non-terminal A in some sentential form. FOLLOW(A) never contains ϵ .
- **Important Formulas/Rules:**
 1. **FIRST for a terminal a:** $\text{FIRST}(a) = \{a\}$.
 2. **FIRST for ϵ :** $\text{FIRST}(\epsilon) = \{\epsilon\}$.
 3. **FIRST for a non-terminal A:** For each production $A \rightarrow X_1 X_2 \dots X_k$:
 - Add $\text{FIRST}(X_1) - \{\epsilon\}$ to $\text{FIRST}(A)$.
 - If $\epsilon \in \text{FIRST}(X_i)$ for all $i = 1 \dots j-1$, then add $\text{FIRST}(X_j) - \{\epsilon\}$ to $\text{FIRST}(A)$.
 - If $\epsilon \in \text{FIRST}(X_i)$ for all $i = 1 \dots k$, then add ϵ to $\text{FIRST}(A)$.
 4. **FOLLOW for a non-terminal A:**
 - Place $\$$ (end-marker) in FOLLOW(S) where S is the start symbol.
 - For each production $A \rightarrow \alpha B \beta$: Add $\text{FIRST}(\beta) - \{\epsilon\}$ to FOLLOW(B).
 - For each production $A \rightarrow \alpha B$ or $A \rightarrow \alpha B \beta$ where $\epsilon \in \text{FIRST}(\beta)$: Add FOLLOW(A) to FOLLOW(B).
- **Common Pitfalls:** Incorrectly handling ϵ productions, or not iterating until sets stabilize.
- **Standard Problem-Solving Technique:** Apply rules iteratively until no new terminals can be added to any set.

Grammar

A grammar is a formal system that describes the syntax of a language. It consists of a set of terminal symbols (the basic symbols of the language), non-terminal symbols (syntactic variables), a start symbol, and a set of production rules that specify how non-terminals can be replaced by sequences of terminals and non-terminals.

- **Formal Definition:** A Context-Free Grammar (CFG) is a 4-tuple $G = (V, T, P, S)$, where:
 - V: Finite set of non-terminal symbols.
 - T: Finite set of terminal symbols ($V \cap T = \emptyset$).
 - P: Finite set of production rules of the form $A \rightarrow \alpha$, where $A \in V$ and $\alpha \in (V \cup T)^*$.
 - $\backslash \text{text}$

2.1

Abstract Syntax Tree (1)

2.1.1 Abstract Syntax Tree: GATE CSE 2015 | Set 2 | Question: 14



In the context of abstract-syntax-tree (AST) and control-flow-graph (CFG), which one of the following is TRUE?

- A. In both AST and CFG, let node N_2 be the successor of node N_1 . In the input program, the code corresponding to N_2 is present after the code corresponding to N_1
- B. For any input program, neither AST nor CFG will contain a cycle
- C. The maximum number of successors of a node in an AST and a CFG depends on the input program
- D. Each node in AST and CFG corresponds to at most one statement in the input program

gatecse-2015-set2 compiler-design easy abstract-syntax-tree code-optimization

Answer key

2.2 Ambiguous Grammar (2)

2.2.1 Ambiguous Grammar: GATE CSE 1987 | Question: 1-xii

A context-free grammar is ambiguous if:

- A. The grammar contains useless non-terminals.
- B. It produces more than one parse tree for some sentence.
- C. Some production has two non terminals side by side on the right-hand side.
- D. None of the above.

gate1987 compiler-design parsing ambiguous-grammar

Answer key

2.2.2 Ambiguous Grammar: GATE CSE 2026 | Set 2 | Question: 19

Which of the following grammars is/are ambiguous?

- A. $S \rightarrow aSb \mid \epsilon$
- B. $E \rightarrow E + E \mid E * E \mid id$
- C. $S \rightarrow aS \mid Sa \mid \epsilon$
- D. $S \rightarrow aS \mid \epsilon$

gatecse-2026-set2 compiler-design ambiguous-grammar multiple-selects one-mark

Answer key

2.3 Assembler (9)

2.3.1 Assembler: GATE CSE 1992 | Question: 01,viii

The purpose of instruction location counter in an assembler is _____

gate1992 compiler-design assembler normal fill-in-the-blanks

Answer key

2.3.2 Assembler: GATE CSE 1992 | Question: 03,ii

Mention the pass number for each of the following activities that occur in a two pass assembler:

- A. object code generation
- B. literals added to literal table
- C. listing printed
- D. address resolution of local symbols

gate1992 compiler-design assembler easy

Answer key

2.3.3 Assembler: GATE CSE 1992 | Question: 3,i

Write short answers to the following:

- i. Which of the following macros can put a macro assembler into an infinite loop?

<pre> .MACRO M1,X .IF EQ,X M1 X+1 .ENDC .IF NE,X .WORD X .ENDC .ENDM </pre>	<pre> .MACRO M2,X .IF EQ,X M2 X .ENDC .IF NE,X .WORD X+1 .ENDC .ENDM </pre>
---	---

Give an example call that does so.

gate1992 compiler-design assembler normal descriptive

Answer key

2.3.4 Assembler: GATE CSE 1993 | Question: 7.6



A simple two-pass assembler does the following in the first pass:

- A. It allocates space for the literals.
- B. It computes the total length of the program.
- C. It builds the symbol table for the symbols and their values.
- D. It generates code for all the load and store register instructions.
- E. None of the above.

gate1993 compiler-design assembler easy multiple-selects

Answer key

2.3.5 Assembler: GATE CSE 1994 | Question: 17a



State whether the following statements are True or False with reasons for your answer:

Coroutine is just another name for a subroutine.

gate1994 compiler-design normal assembler true-false descriptive

Answer key

2.3.6 Assembler: GATE CSE 1994 | Question: 17b



State whether the following statements are True or False with reasons for your answer:

A two pass assembler uses its machine opcode table in the first pass of assembly.

gate1994 compiler-design normal assembler true-false descriptive

Answer key

2.3.7 Assembler: GATE CSE 1994 | Question: 18a



State whether the following statements are True or False with reasons for your answer

A subroutine cannot always be used to replace a macro in an assembly language program.

gate1994 compiler-design normal assembler true-false descriptive

Answer key

2.3.8 Assembler: GATE CSE 1994 | Question: 18b



State whether the following statements are True or False with reasons for your answer

A symbol declared as 'external' in an assembly language program is assigned an address outside the program by the assembler itself.

gate1994 compiler-design normal assembler true-false descriptive

2.3.9 Assembler: GATE CSE 1996 | Question: 1.17



The pass numbers for each of the following activities

- object code generation
- literals added to literal table
- listing printed
- address resolution of local symbols that occur in a two pass assembler

respectively are

- A. 1,2,1,2 B. 2,1,2,1 C. 2,1,1,2 D. 1,2,2,2

gate1996 compiler-design normal assembler

2.4

Backpatching (1)

2.4.1 Backpatching: GATE CSE 2025 | Set 2 | Question: 11



Consider the following statements about the use of backpatching in a compiler for intermediate code generation:

- Backpatching can be used to generate code for Boolean expression in one pass.
- Backpatching can be used to generate code for flow-of-control statements in one pass.

Which ONE of the following options is CORRECT?

- A. Only I is correct B. Only II is correct
C. Both I and II are correct D. Neither I nor II is correct

gatecse2025-set2 compiler-design backpatching intermediate-code one-mark

2.5

Basic Blocks (2)

2.5.1 Basic Blocks: GATE CSE 2025 | Set 1 | Question: 42



Refer to the given 3-address code sequence. This code sequence is split into basic blocks. The number of basic blocks is _____. (Answer in integer)

```

1001: i = 1
1002: j = 1
1003: t1 = 10*i
1004: t2 = t1+j
1005: t3 = 8*t2
1006: t4 = t3-88
1007: a[t4] = 0.0
1008: j = j+1
1009: if j <= 10 goto 1003
1010: i = i+1
1011: if i <= 10 goto 1002
1012: i = 1
1013: t5 = i-1
1014: t6 = 88*t5
1015: a[t6] = 1.0
1016: i = i+1
1017: if i <= 10 goto 1013

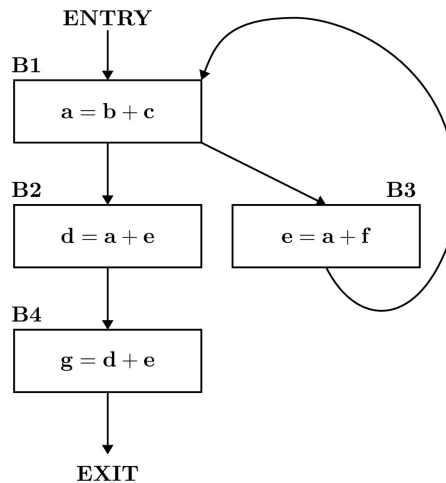
```

gatecse2025-set1 compiler-design three-address-code basic-blocks numerical-answers easy two-marks

2.5.2 Basic Blocks: GATE CSE 2026 | Set 2 | Question: 35



Consider the control flow graph given below.



Which one of the following options is the set of live variables at the exit point of each basic block?

- A. B1 : {a, b, c, e, f}, B2 : {d, e}, B3 : {b, c, e, f}, B4 : $\emptyset$
- B. B1: $\emptyset$, B2: {d, e}, B3: {a, c, f}, B4: $\emptyset$
- C. B1 : {a, b, c, e, f}, B2 : {d, e}, B3 : {c, e, f}, B4 : $\emptyset$
- D. B1 : $\emptyset$, B2 : {d, e, f}, B3 : {a, b, c, e, f}, B4 : $\emptyset$

gatecse-2026-set2 compiler-design basic-blocks two-marks

Answer key

2.6

Code Optimization (8)

Practice Test: Test 1 (13Q)

2.6.1 Code Optimization: GATE CSE 2006 | Question: 55



Consider these two functions and two statements S1 and S2 about them.

<pre> int work1(int *a, int i, int j) { int x = a[i+2]; a[j] = x+1; return a[i+2] - 3; } </pre>	<pre> int work2(int *a, int i, int j) { int t1 = i+2; int t2 = a[t1]; a[j] = t2+1; return t2 - 3; } </pre>
---	--

S1: The transformation from *work1* to *work2* is valid, i.e., for any program state and input arguments, *work2* will compute the same output and have the same effect on program state as *work1*

S2: All the transformations applied to *work1* to get *work2* will always improve the performance (i.e reduce CPU time) of *work2* compared to *work1*

- A. S1 is false and S2 is false
- B. S1 is false and S2 is true
- C. S1 is true and S2 is false
- D. S1 is true and S2 is true

gatecse-2006 compiler-design normal code-optimization

Answer key

2.6.2 Code Optimization: GATE CSE 2006 | Question: 60



Consider the following C code segment.

```

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)

```



```

{
  if (i%2)
  {
    x += (4*i + 5*i);
    y += (7 + 4*i);
  }
}

```

Which one of the following is false?

- A. The code contains loop invariant computation
- B. There is scope of common sub-expression elimination in this code
- C. There is scope of strength reduction in this code
- D. There is scope of dead code elimination in this code

gatecse-2006 compiler-design code-optimization

Answer key

2.6.3 Code Optimization: GATE CSE 2008 | Question: 12



Some code optimizations are carried out on the intermediate code because

- A. They enhance the portability of the compiler to the target processor
- B. Program analysis is more accurate on intermediate code than on machine code
- C. The information from dataflow analysis cannot otherwise be used for optimization
- D. The information from the front end cannot otherwise be used for optimization

gatecse-2008 normal code-optimization compiler-design

Answer key

2.6.4 Code Optimization: GATE CSE 2014 | Set 1 | Question: 17



Which one of the following is **FALSE**?

- A. A basic block is a sequence of instructions where control enters the sequence at the beginning and exits at the end.
- B. Available expression analysis can be used for common subexpression elimination.
- C. Live variable analysis can be used for dead code elimination.
- D. $x = 4 * 5 \Rightarrow x = 20$ is an example of common subexpression elimination.

gatecse-2014-set1 compiler-design code-optimization normal

Answer key

2.6.5 Code Optimization: GATE CSE 2014 | Set 3 | Question: 11



The minimum number of arithmetic operations required to evaluate the polynomial $P(X) = X^5 + 4X^3 + 6X + 5$ for a given value of X , using only one temporary variable is _____.

gatecse-2014-set3 compiler-design numerical-answers normal code-optimization

Answer key

2.6.6 Code Optimization: GATE CSE 2021 | Set 2 | Question: 30



Consider the following ANSI C code segment:

```

z = x + 3 + y ->f1 + y ->f2;
for (i = 0; i < 200; i = i + 2)
{
  if (z > i)
  {
    p = p + x + 3;
    q = q + y ->f1;
  } else
  {

```

```

p = p + y->f2;
q = q + x + 3;
}

```

Assume that the variable y points to a **struct** (allocated on the heap) containing two fields $f1$ and $f2$, and the local variables x, y, z, p, q , and i are allotted registers. Common sub-expression elimination (CSE) optimization is applied on the code. The number of addition and the dereference operations (of the form $y \rightarrow f1$ or $y \rightarrow f2$) in the optimized code, respectively, are:

- A. 403 and 102 B. 203 and 2 C. 303 and 102 D. 303 and 2

gatecse-2021-set2 code-optimization compiler-design two-marks

Answer key

2.6.7 Code Optimization: GATE CSE 2025 | Set 1 | Question: 3



Which ONE of the following techniques used in compiler code optimization uses live variable analysis?

- A. Run-time function call management B. Register assignment to variables
C. Strength reduction D. Constant folding

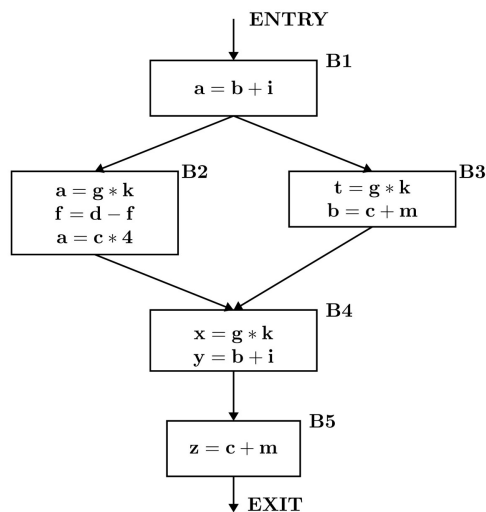
gatecse2025-set1 compiler-design code-optimization easy one-mark

Answer key

2.6.8 Code Optimization: GATE CSE 2026 | Set 1 | Question: 32



Consider the control flow graph shown in the figure.



Which one of the following options correctly lists the set of redundant expressions (common subexpressions) in the basic blocks B4 and B5?

Note: All the variables are integers.

- A. B4: $\{b + i\}$ B5: $\{c + m\}$
B. B4: $\{g * k\}$ B5: $\{c + m\}$
C. B4: $\{g * k, b + i\}$ B5: $\{\}$
D. B4: $\{g * k\}$ B5: $\{\}$

gatecse-2026-set1 compiler-design two-marks code-optimization

Answer key

2.7.1 Compilation Phases: GATE CSE 1987 | Question: 1-xi



In a compiler the module that checks every character of the source text is called:

- A. The code generator.
- B. The code optimiser.
- C. The lexical analyser.
- D. The syntax analyser.

gate1987 compiler-design compilation-phases lexical-analysis

Answer key

2.7.2 Compilation Phases: GATE CSE 1990 | Question: 2-ix



Match the pairs in the following questions:

(a) Lexical analysis	(p) DAG's
(b) Code optimization	(q) Syntax trees
(c) Code generation	(r) Push down automaton
(d) Abelian groups	(s) Finite automaton

gate1990 match-the-following compiler-design compilation-phases

Answer key

2.7.3 Compilation Phases: GATE CSE 2005 | Question: 61



Consider line number 3 of the following C-program.

```
int main() { /*Line 1 */  
    int i, N; /*Line 2 */  
    for (i=0, i<N, i++); /*Line 3 */  
}
```

Identify the compiler's response about this line while creating the object-module:

- A. No compilation error
- B. Only a lexical error
- C. Only syntactic errors
- D. Both lexical and syntactic errors

gatecse-2005 compiler-design compilation-phases normal

Answer key

2.7.4 Compilation Phases: GATE CSE 2009 | Question: 17



Match all items in Group 1 with the correct options from those given in Group 2.

Group 1	Group 2
P. Regular Expression	1. Syntax analysis
Q. Pushdown automata	2. Code generation
R. Dataflow analysis	3. Lexical analysis
S. Register allocation	4. Code optimization

- A. P-4, Q-1, R-2, S-3
- B. P-3, Q-1, R-4, S-2
- C. P-3, Q-4, R-1, S-2
- D. P-2, Q-1, R-4, S-3

gatecse-2009 compiler-design easy compilation-phases match-the-following

Answer key

2.7.5 Compilation Phases: GATE CSE 2015 | Set 2 | Question: 19



Match the following:

P. Lexical analysis	1. Graph coloring
Q. Parsing	2. DFA minimization
R. Register allocation	3. Post-order traversal
S. Expression evaluation	4. Production tree

- A. P-2, Q-3, R-1, S-4
C. P-2, Q-4, R-1, S-3

- B. P-2, Q-1, R-4, S-3
D. P-2, Q-3, R-4, S-1

gatecse-2015-set2 compiler-design normal compilation-phases match-the-following

Answer key

2.7.6 Compilation Phases: GATE CSE 2016 | Set 2 | Question: 19



Match the following:

(P) Lexical analysis	(i) Leftmost derivation
(Q) Top down parsing	(ii) Type checking
(R) Semantic analysis	(iii) Regular expressions
(S) Runtime environment	(iv) Activation records

- A. $P \leftrightarrow i, Q \leftrightarrow ii, R \leftrightarrow iv, S \leftrightarrow iii$
C. $P \leftrightarrow ii, Q \leftrightarrow iii, R \leftrightarrow i, S \leftrightarrow iv$

- B. $P \leftrightarrow iii, Q \leftrightarrow i, R \leftrightarrow ii, S \leftrightarrow iv$
D. $P \leftrightarrow iv, Q \leftrightarrow i, R \leftrightarrow ii, S \leftrightarrow iii$

gatecse-2016-set2 compiler-design easy match-the-following compilation-phases

Answer key

2.7.7 Compilation Phases: GATE CSE 2017 | Set 2 | Question: 05



Match the following according to input (from the left column) to the compiler phase (in the right column) that processes it:

P. Syntax tree	i. Code generator
Q. Character stream	ii. Syntax analyser
R. Intermediate representation	iii. Semantic analyser
S. Token stream	iv. Lexical analyser

- A. P-ii; Q-iii; R-iv; S-i
C. P-iii; Q-iv; R-i; S-ii

- B. P-ii; Q-i; R-iii; S-iv
D. P-i; Q-iv; R-ii; S-iii

gatecse-2017-set2 compiler-design match-the-following compilation-phases easy

Answer key

2.7.8 Compilation Phases: GATE CSE 2018 | Question: 8



Which one of the following statements is FALSE?

- A. Context-free grammar can be used to specify both lexical and syntax rules
B. Type checking is done before parsing
C. High-level language programs can be translated to different Intermediate Representations
D. Arguments to a function can be passed using the program stack

gatecse-2018 compiler-design easy compilation-phases one-mark

Answer key

2.7.9 Compilation Phases: GATE CSE 2020 | Question: 9



Consider the following statements.

- I. Symbol table is accessed only during lexical analysis and syntax analysis.
- II. Compilers for programming languages that support recursion necessarily need heap storage for memory allocation in the run-time environment.

III. Errors violating the condition '*any variable must be declared before its use*' are detected during syntax analysis.

Which of the above statements is/are TRUE?

- A. I only
- B. I and III only
- C. II only
- D. None of I, II and III

gatecse-2020 compiler-design compilation-phases runtime-environment one-mark

Answer key

2.7.10 Compilation Phases: GATE CSE 2021 | Set 2 | Question: 3



Consider the following ANSI C program:

```
int main () {  
    Integer x;  
    return 0;  
}
```

Which one of the following phases in a seven-phase C compiler will throw an error?

- A. Lexical analyzer
- B. Syntax analyzer
- C. Semantic analyzer
- D. Machine dependent optimizer

gatecse-2021-set2 compilation-phases compiler-design one-mark

Answer key

2.7.11 Compilation Phases: GATE CSE 2023 | Question: 1



Consider the following statements regarding the front-end and back-end of a compiler.

S1: The front-end includes phases that are independent of the target hardware.

S2: The back-end includes phases that are specific to the target hardware.

S3: The back-end includes phases that are specific to the programming language used in the source code.

Identify the CORRECT option.

- A. Only **S1** is TRUE.
- B. Only **S1** and **S2** are TRUE.
- C. **S1**, **S2**, and **S3** are all TRUE.
- D. Only **S1** and **S3** are TRUE.

gatecse-2023 compiler-design compilation-phases one-mark

Answer key

2.7.12 Compilation Phases: GATE CSE 2024 | Set 2 | Question: 11



Consider the following two sets:

Set X	Set Y
P. Lexical Analyzer	1. Abstract Syntax Tree
Q. Syntax Analyzer	2. Token
R. Intermediate Code Generator	3. Parse Tree
S. Code Optimizer	4. Constant Folding

Which one of the following options is the CORRECT match from **Set X to Set Y**?

- A. P – 4; Q – 1; R – 3; S – 2
- B. P – 2; Q – 3; R – 1; S – 4
- C. P – 2; Q – 1; R – 3; S – 4
- D. P – 4; Q – 3; R – 2; S – 1

gatecse-2024-set2 compiler-design compilation-phases match-the-following easy one-mark

Answer key

2.7.13 Compilation Phases: GATE CSE 2026 | Set 1 | Question: 17



Consider the following C statements:

```
char *str1 = "Hello; /* Statement S1 */
char *str2 = "Hello;"; /* Statement S2 */
int *str3 = "Hello"; /* Statement S3 */
```

Which of the following options is/are correct?

- A. S1 and S2 have syntactic errors
- B. S2 has a lexical error and S3 has a syntactic error
- C. S1 has a lexical error and S3 has a semantic error
- D. S1 has a syntactic error and S3 has a semantic error

gatecse-2026-set1 compiler-design multiple-selects one-mark compilation-phases

Answer key

2.8

Compiler tokenization (1)

2.8.1 Compiler tokenization: GATE CSE 2026 | Set 2 | Question: 25



A lexical analyzer uses the following token definitions

- $letter \rightarrow [A - Za - z]$
- $digit \rightarrow [0 - 9]$
- $id \rightarrow letter(letter|digit)^*$
- $number \rightarrow digit^+$
- $ws \rightarrow (blank|tab|newline)^+$

For the string given below,

$x1$ 23 mm 78 y 7z zz5 14A 8H AaYcD

the number of tokens (excluding ws) that will be produced by the lexical analyzer is _____. (answer in integer)

gatecse-2026-set2 compiler-design compiler-tokenization numerical-answers one-mark

Answer key

2.9

Directed Acyclic Graph (2)

2.9.1 Directed Acyclic Graph: GATE CSE 2014 | Set 3 | Question: 34



Consider the basic block given below.

```
a = b + c
c = a + d
d = b + c
e = d - b
a = e + b
```

The minimum number of nodes and edges present in the DAG representation of the above basic block respectively are

- A. 6 and 6
- B. 8 and 10
- C. 9 and 12
- D. 4 and 4

gatecse-2014-set3 compiler-design code-optimization directed-acyclic-graph normal

Answer key

2.9.2 Directed Acyclic Graph: GATE CSE 2021 | Set 1 | Question: 50



Consider the following C code segment:

```
a = b + c;
e = a + 1;
```

```

d = b + c;
f = d + 1;
g = e + f;

```

In a compiler, this code segment is represented internally as a directed acyclic graph (DAG). The number of nodes in the DAG is _____

gatecse-2021-set1 compiler-design code-optimization directed-acyclic-graph numerical-answers two-marks

Answer key

2.10

Expression Evaluation (2)

2.10.1 Expression Evaluation: GATE CSE 2002 | Question: 2.19



To evaluate an expression without any embedded function calls

- One stack is enough
- Two stacks are needed
- As many stacks as the height of the expression tree are needed
- A Turing machine is needed in the general case

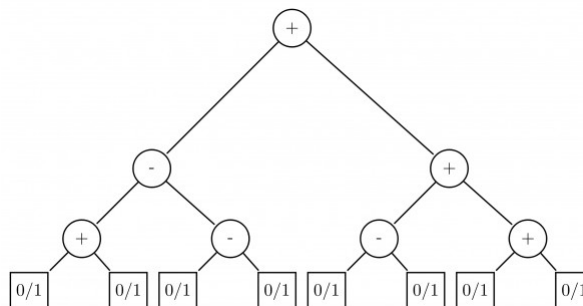
gatecse-2002 compiler-design expression-evaluation easy

Answer key

2.10.2 Expression Evaluation: GATE CSE 2014 | Set 2 | Question: 39



Consider the expression tree shown. Each leaf represents a numerical value, which can either be 0 or 1. Over all possible choices of the values at the leaves, the maximum possible value of the expression represented by the tree is ____.



gatecse-2014-set2 compiler-design normal expression-evaluation numerical-answers

Answer key

2.11

First and Follow (6)

Practice Test: Test 1 (12Q)

2.11.1 First and Follow: GATE CSE 1992 | Question: 02,xiii



For a context free grammar, FOLLOW(A) is the set of terminals that can appear immediately to the right of non-terminal A in some "sentential" form. We define two sets LFOLLOW(A) and RFOLLOW(A) by replacing the word "sentential" by "left sentential" and "right most sentential" respectively in the definition of FOLLOW (A).

- | | |
|---|--|
| A. FOLLOW(A) and LFOLLOW(A) may be different. | B. FOLLOW(A) and RFOLLOW(A) are always the same. |
| C. All the three sets are identical. | D. All the three sets are different. |

gate1992 parsing compiler-design normal multiple-selects first-and-follow

Answer key

2.11.2 First and Follow: GATE CSE 2012 | Question: 52



For the grammar below, a partial LL(1) parsing table is also presented along with the grammar. Entries that

need to be filled are indicated as $E1$, $E2$, and $E3$. ϵ is the empty string, $\$$ indicates end of input, and, | separates alternate right hand sides of productions.

- $S \rightarrow aAbB \mid bAaB \mid \epsilon$
- $A \rightarrow S$
- $B \rightarrow S$

	a	b	\$
S	E1	E2	$S \rightarrow \epsilon$
A	$A \rightarrow S$	$A \rightarrow S$	error
B	$B \rightarrow S$	$B \rightarrow S$	E3

The FIRST and FOLLOW sets for the non-terminals A and B are

- A. $\text{FIRST}(A) = \{a, b, \epsilon\} = \text{FIRST}(B)$
 $\text{FOLLOW}(A) = \{a, b\}$
 $\text{FOLLOW}(B) = \{a, b, \$\}$
- B. $\text{FIRST}(A) = \{a, b, \$\}$
 $\text{FIRST}(B) = \{a, b, \epsilon\}$
 $\text{FOLLOW}(A) = \{a, b\}$
 $\text{FOLLOW}(B) = \{\$\}$
- C. $\text{FIRST}(A) = \{a, b, \epsilon\} = \text{FIRST}(B)$
 $\text{FOLLOW}(A) = \{a, b\}$
 $\text{FOLLOW}(B) = \emptyset$
- D. $\text{FIRST}(A) = \{a, b\} = \text{FIRST}(B)$
 $\text{FOLLOW}(A) = \{a, b\}$
 $\text{FOLLOW}(B) = \{a, b\}$

gatecse-2012 compiler-design parsing normal first-and-follow

Answer key

2.11.3 First and Follow: GATE CSE 2017 | Set 1 | Question: 17



Consider the following grammar:

- $P \rightarrow xQRS$
- $Q \rightarrow yz \mid z$
- $R \rightarrow w \mid \epsilon$
- $S \rightarrow y$

What is $\text{FOLLOW}(Q)$?

- A. $\{R\}$ B. $\{w\}$ C. $\{w, y\}$ D. $\{w, \$\}$

gatecse-2017-set1 compiler-design parsing easy first-and-follow

Answer key

2.11.4 First and Follow: GATE CSE 2019 | Question: 19



Consider the grammar given below:

- $S \rightarrow Aa$
- $A \rightarrow BD$
- $B \rightarrow b \mid \epsilon$
- $D \rightarrow d \mid \epsilon$

Let a, b, d and $\$$ be indexed as follows:

a	b	d	$\$$
3	2	1	0

Compute the FOLLOW set of the non-terminal B and write the index values for the symbols in the FOLLOW set in the descending order. (For example, if the FOLLOW set is $(a, b, d, \$)$, then the answer should be 3210)

gatecse-2019 numerical-answers compiler-design parsing one-mark first-and-follow

Answer key

2.11.5 First and Follow: GATE CSE 2024 | Set 1 | Question: 28



Consider the following grammar G , with S as the start symbol. The grammar G has three incomplete productions denoted by (1), (2), and (3).

$$S \rightarrow daT \mid \quad (1)$$

$$T \rightarrow aS|bT| \quad (2)$$

$$R \rightarrow (3) \mid \epsilon$$

The set of terminals is $\{a, b, c, d, f\}$. The FIRST and FOLLOW sets of the different non-terminals are as follows.

$$\text{FIRST}(S) = \{c, d, f\}, \text{FIRST}(T) = \{a, b, \epsilon\}, \text{FIRST}(R) = \{c, \epsilon\}$$

$$\text{FOLLOW}(S) = \text{FOLLOW}(T) = \{c, f, \$\}, \text{FOLLOW}(R) = \{f\}$$

Which one of the following options CORRECTLY fills in the incomplete productions?

- A. (1) $S \rightarrow Rf$ (2) $T \rightarrow \epsilon$ (3) $R \rightarrow cTR$
- B. (1) $S \rightarrow fR$ (2) $T \rightarrow \epsilon$ (3) $R \rightarrow cTR$
- C. (1) $S \rightarrow fR$ (2) $T \rightarrow cT$ (3) $R \rightarrow cR$
- D. (1) $S \rightarrow Rf$ (2) $T \rightarrow cT$ (3) $R \rightarrow cR$

gatecse-2024-set1 compiler-design first-and-follow two-marks

Answer key

2.11.6 First and Follow: GATE CSE 2025 | Set 1 | Question: 36



Which of the following statement(s) is/are TRUE while computing First and Follow during top down parsing by a compiler?

- A. For a production $A \rightarrow \epsilon$, ϵ will be added to $\text{First}(A)$.
- B. If there is any input right end marker, it will be added to $\text{First}(S)$, where S is the start symbol.
- C. For a production $A \rightarrow \epsilon$, ϵ will be added to $\text{Follow}(A)$.
- D. If there is any input right end marker, it will be added to $\text{Follow}(S)$, where S is the start symbol.

gatecse2025-set1 compiler-design first-and-follow parsing multiple-selects two-marks

Answer key

2.12 Grammar (47)

Practice Tests: Test 1 (15Q) Test 2 (15Q) Test 3 (6Q)

2.12.1 Grammar: GATE CSE 1988 | Question: 10ia



Consider the following grammar:

- $S \rightarrow S$
- $S \rightarrow SS \mid a \mid \epsilon$

Construct the collection of sets of LR (0) items for this grammar and draw its goto graph.

Answer key

2.12.2 Grammar: GATE CSE 1988 | Question: 10ib



Consider the following grammar:

- $S \rightarrow S$
- $S \rightarrow SS \mid a \mid \epsilon$

Indicate the shift-reduce and reduce-reduce conflict (if any) in the various states of the LR(0) parser.

Answer key

2.12.3 Grammar: GATE CSE 1990 | Question: 16a



Show that grammar G_1 is ambiguous using parse trees:

$G_1 : S \rightarrow \text{if } S \text{ then } S \text{ else } S$

$S \rightarrow \text{if } S \text{ then } S$

Answer key

2.12.4 Grammar: GATE CSE 1991 | Question: 10b



Consider the following grammar for arithmetic expressions using binary operators $-$ and $/$ which are not associative

- $E \rightarrow E - T \mid T$
- $T \rightarrow T / F \mid F$
- $F \rightarrow (E) \mid id$

(E is the start symbol)

Does the grammar allow expressions with redundant parentheses as in (id/id) or in $id - (id/id)$? If so, convert the grammar into one which does not generate expressions with redundant parentheses. Do this with minimum number of changes to the given production rules and adding at most one more production rule.

Answer key

2.12.5 Grammar: GATE CSE 1991 | Question: 10c



Consider the following grammar for arithmetic expressions using binary operators $-$ and $/$ which are not associative

- $E \rightarrow E - T \mid T$
- $T \rightarrow T / F \mid F$
- $F \rightarrow (E) \mid id$

(E is the start symbol)

Does the grammar allow expressions with redundant parentheses as in (id/id) or in $id - (id/id)$? If so, convert the grammar into one which does not generate expressions with redundant parentheses. Do this with minimum number of changes to the given production rules and adding at most one more production rule.

Convert the grammar obtained above into one that is not left recursive.

Answer key

2.12.6 Grammar: GATE CSE 1992 | Question: 02,xviii



If G is a context free grammar and w is a string of length l in $L(G)$, how long is a derivation of w in G , if G is in Chomsky normal form?

- A. $2l$ B. $2l + 1$ C. $2l - 1$ D. l

gate1992 compiler-design easy grammar

Answer key

2.12.7 Grammar: GATE CSE 1994 | Question: 1.18



Which of the following features cannot be captured by context-free grammars?

- A. Syntax of if-then-else statements B. Syntax of recursive procedures
C. Whether a variable has been declared before its use D. Variable names of arbitrary length

gate1994 compiler-design grammar normal

Answer key

2.12.8 Grammar: GATE CSE 1994 | Question: 20



A grammar G is in Chomsky-Normal Form (CNF) if all its productions are of the form $A \rightarrow BC$ or $A \rightarrow a$, where A, B and C , are non-terminals and a is a terminal. Suppose G is a CFG in CNF and w is a string in $L(G)$ of length n , then how long is a derivation of w in G ?

gate1994 compiler-design grammar normal descriptive

Answer key

2.12.9 Grammar: GATE CSE 1994 | Question: 3.5



Match the following items

(i) Backus-Naur form	(a) Regular expressions
(ii) Lexical analysis	(b) LALR(1) grammar
(iii) YACC	(c) LL(1) grammars
(iv) Recursive descent parsing	(d) General context-free grammars

gate1994 compiler-design grammar normal match-the-following

Answer key

2.12.10 Grammar: GATE CSE 1995 | Question: 1.10



Consider a grammar with the following productions

- $S \rightarrow a\alpha b \mid b\alpha c \mid aB$
- $S \rightarrow \alpha S \mid b$
- $S \rightarrow \alpha bb \mid ab$
- $S\alpha \rightarrow bdb \mid b$

The above grammar is:

- A. Context free B. Regular C. Context sensitive D. $LR(k)$

gate1995 compiler-design grammar normal

Answer key

2.12.11 Grammar: GATE CSE 1995 | Question: 9



- A. Translate the arithmetic expression $a^* - (b + c)$ into syntax tree.
- B. A grammar is said to have cycles if it is the case that $A \xRightarrow{+} A$
Show that no grammar that has cycles can be LL(1).

gate1995 compiler-design grammar normal descriptive

Answer key

2.12.12 Grammar: GATE CSE 1996 | Question: 11



Let G be a context-free grammar where $G = (\{S, A, B, C\}, \{a, b, d\}, P, S)$ with the productions in P given below.

- $S \rightarrow ABAC$
- $A \rightarrow aA \mid \varepsilon$
- $B \rightarrow bB \mid \varepsilon$
- $C \rightarrow d$

(ε denotes the null string). Transform the grammar G to an equivalent context-free grammar G' that has no ε productions and no unit productions. (A unit production is of the form $x \rightarrow y$, and x and y are non terminals).

gate1996 compiler-design grammar normal descriptive

Answer key

2.12.13 Grammar: GATE CSE 1996 | Question: 2.10



The grammar whose productions are

- $\langle \text{stmt} \rangle \rightarrow \text{if id then } \langle \text{stmt} \rangle$
- $\langle \text{stmt} \rangle \rightarrow \text{if id then } \langle \text{stmt} \rangle \text{ else } \langle \text{stmt} \rangle$
- $\langle \text{stmt} \rangle \rightarrow \text{id} := \text{id}$

is ambiguous because

(a) the sentence

if a then if b then c := d

has more than two parse trees

(b) the left most and right most derivations of the sentence

if a then if b then c := d

give rise to different parse trees

(c) the sentence

if a then if b then c := d else c := f

has more than two parse trees

(d) the sentence

if a then if b then c := d else c := f

has two parse trees

gate1996 compiler-design grammar normal

Answer key

2.12.14 Grammar: GATE CSE 1997 | Question: 11



Consider the grammar

- $S \rightarrow bSe$
- $S \rightarrow PQR$

- $P \rightarrow bPc$
- $P \rightarrow \varepsilon$
- $Q \rightarrow cQd$
- $Q \rightarrow \varepsilon$
- $R \rightarrow dRe$
- $R \rightarrow \varepsilon$

where S, P, Q, R are non-terminal symbols with S being the start symbol; b, c, d, e are terminal symbols and ' ε ' is the empty string. This grammar generates strings of the form b^i, c^j, d^k, e^m for some $i, j, k, m \geq 0$.

- What is the condition on the values of i, j, k, m ?
- Find the smallest string that has two parse trees.

gate1997 compiler-design grammar normal theory-of-computation descriptive

Answer key

2.12.15 Grammar: GATE CSE 1998 | Question: 14



A. Let $G_1 = (N, T, P, S_1)$ be a CFG where, $N = \{S_1, A, B\}$, $T = \{a, b\}$ and P is given by

$$\begin{array}{l|l} S_1 \rightarrow aS_1b & S_1 \rightarrow aBb \\ S_1 \rightarrow aAb & B \rightarrow Bb \\ A \rightarrow aA & B \rightarrow b \\ A \rightarrow a & \end{array}$$

What is $L(G_1)$?

- Use the grammar in Part(a) to give a CFG for $L_2 = \{a^i b^j a^k b^l \mid i, j, k, l \geq 1, i = j \text{ or } k = l\}$ by adding not more than 5 production rules.
- Is L_2 inherently ambiguous?

gate1998 compiler-design grammar descriptive

Answer key

2.12.16 Grammar: GATE CSE 1998 | Question: 6b



Consider the grammar

- $S \rightarrow Aa \mid b$
- $A \rightarrow Ac \mid Sd \mid \varepsilon$

Construct an equivalent grammar with no left recursion and with minimum number of production rules.

gate1998 compiler-design grammar descriptive

Answer key

2.12.17 Grammar: GATE CSE 1999 | Question: 2.15



A grammar that is both left and right recursive for a non-terminal, is

- Ambiguous
- Unambiguous
- Information is not sufficient to decide whether it is ambiguous or unambiguous
- None of the above

gate1999 compiler-design grammar normal

Answer key

2.12.18 Grammar: GATE CSE 2001 | Question: 1.18



Which of the following statements is false?

- A. An unambiguous grammar has same leftmost and rightmost derivation
- B. An LL(1) parser is a top-down parser
- C. LALR is more powerful than SLR
- D. An ambiguous grammar can never be LR(k) for any k

gatecse-2001 compiler-design grammar normal

Answer key

2.12.19 Grammar: GATE CSE 2001 | Question: 18



- A. Remove left-recursion from the following grammar: $S \rightarrow Sa \mid Sb \mid a \mid b$
- B. Consider the following grammar:

$$S \rightarrow aSbS \mid bSaS \mid \epsilon$$

Construct all possible parse trees for the string abab. Is the grammar ambiguous?

gatecse-2001 compiler-design grammar descriptive

Answer key

2.12.20 Grammar: GATE CSE 2003 | Question: 56



Consider the grammar shown below

- $S \rightarrow iEtSS' \mid a$
- $S' \rightarrow eS \mid \epsilon$
- $E \rightarrow b$

In the predictive parse table, M , of this grammar, the entries $M[S', e]$ and $M[S', \$]$ respectively are

- A. $\{S' \rightarrow eS\}$ and $\{S' \rightarrow \epsilon\}$
- B. $\{S' \rightarrow eS\}$ and $\{\}$
- C. $\{S' \rightarrow \epsilon\}$ and $\{S' \rightarrow \epsilon\}$
- D. $\{S' \rightarrow eS, S' \rightarrow \epsilon\}$ and $\{S' \rightarrow \epsilon\}$

gatecse-2003 compiler-design grammar normal parsing

Answer key

2.12.21 Grammar: GATE CSE 2003 | Question: 57



Consider the grammar shown below.

- $S \rightarrow CC$
- $C \rightarrow cC \mid d$

This grammar is

- A. LL(1)
- B. SLR(1) but not LL(1)
- C. LALR(1) but not SLR(1)
- D. LR(1) but not LALR(1)

gatecse-2003 compiler-design grammar parsing normal

Answer key

2.12.22 Grammar: GATE CSE 2003 | Question: 58



Consider the translation scheme shown below.

- $S \rightarrow T R$
- $R \rightarrow +T\{\text{print}(' + '); \} R \mid \epsilon$
- $T \rightarrow \text{num} \{\text{print}(\text{num.val}); \}$

Here **num** is a token that represents an integer and **num.val** represents the corresponding integer value. For an input string '9 + 5 + 2', this translation scheme will print

- A. 9 + 5 + 2 B. 9 5 + 2 + C. 9 5 2 + + D. + + 9 5 2

gatecse-2003 compiler-design grammar normal

Answer key

2.12.23 Grammar: GATE CSE 2004 | Question: 8



Which of the following grammar rules violate the requirements of an operator grammar? P, Q, R are nonterminals, and r, s, t are terminals.

- I. $P \rightarrow QR$
- II. $P \rightarrow QsR$
- III. $P \rightarrow \epsilon$
- IV. $P \rightarrow QtRr$

- A. (I) only B. (I) and (III) only C. (II) and (III) only D. (III) and (IV) only

gatecse-2004 compiler-design grammar normal

Answer key

2.12.24 Grammar: GATE CSE 2004 | Question: 88



Consider the following grammar G:

$$S \rightarrow bS \mid aA \mid b$$

$$A \rightarrow bA \mid aB$$

$$B \rightarrow bB \mid aS \mid a$$

Let $N_a(w)$ and $N_b(w)$ denote the number of a's and b's in a string w respectively.

The language $L(G)$ over $\{a, b\}^+$ generated by G is

- A. $\{w \mid N_a(w) > 3N_b(w)\}$
- B. $\{w \mid N_b(w) > 3N_a(w)\}$
- C. $\{w \mid N_a(w) = 3k, k \in \{0, 1, 2, \dots\}\}$
- D. $\{w \mid N_b(w) = 3k, k \in \{0, 1, 2, \dots\}\}$

gatecse-2004 compiler-design grammar normal

Answer key

2.12.25 Grammar: GATE CSE 2005 | Question: 14



The grammar $A \rightarrow AA \mid (A) \mid \epsilon$ is not suitable for predictive-parsing because the grammar is:

- A. ambiguous B. left-recursive C. right-recursive D. an operator-grammar

gatecse-2005 compiler-design parsing grammar easy

Answer key

2.12.26 Grammar: GATE CSE 2005 | Question: 59



Consider the grammar:

$$E \rightarrow E + n \mid E \times n \mid n$$

For a sentence $n + n \times n$, the handles in the right-sentential form of the reduction are:

- A. $n, E + n$ and $E + n \times n$ B. $n, E + n$ and $E + E \times n$
 C. $n, n + n$ and $n + n \times n$ D. $n, E + n$ and $E \times n$

gatecse-2005 compiler-design grammar normal

Answer key

2.12.27 Grammar: GATE CSE 2006 | Question: 32, ISRO2016-35



Consider the following statements about the context free grammar

$$G = \{S \rightarrow SS, S \rightarrow ab, S \rightarrow ba, S \rightarrow \epsilon\}$$

- I. G is ambiguous
 II. G produces all strings with equal number of a 's and b 's
 III. G can be accepted by a deterministic PDA.

Which combination below expresses all the true statements about G ?

- A. I only B. I and III only C. II and III only D. I, II and III

gatecse-2006 compiler-design grammar normal isro2016

Answer key

2.12.28 Grammar: GATE CSE 2006 | Question: 59



Consider the following translation scheme.

- $S \rightarrow ER$
- $R \rightarrow *E \{ \text{print}(' * '); \} R \mid \epsilon$
- $E \rightarrow F + E \{ \text{print}(' + '); \} \mid F$
- $F \rightarrow (S) \mid id \{ \text{print}(id.value); \}$

Here id is a token that represents an integer and $id.value$ represents the corresponding integer value. For an input ' $2 * 3 + 4$ ', this translation scheme prints

- A. $2 * 3 + 4$ B. $2 * + 3 4$ C. $2 3 * 4 +$ D. $2 3 4 + *$

gatecse-2006 compiler-design grammar normal

Answer key

2.12.29 Grammar: GATE CSE 2006 | Question: 84



Which one of the following grammars generates the language $L = \{a^i b^j \mid i \neq j\}$?

- A. $S \rightarrow AC \mid CB$ B. $S \rightarrow aS \mid Sb \mid a \mid b$ C. $S \rightarrow AC \mid CB$ D. $S \rightarrow AC \mid CB$
 $C \rightarrow aCb \mid a \mid b$ $C \rightarrow aCb \mid \epsilon$ $C \rightarrow aCb \mid \epsilon$
 $A \rightarrow aA \mid \epsilon$ $A \rightarrow aA \mid \epsilon$ $A \rightarrow aA \mid a$
 $B \rightarrow Bb \mid \epsilon$ $B \rightarrow Bb \mid \epsilon$ $B \rightarrow Bb \mid b$

gatecse-2006 compiler-design grammar normal theory-of-computation

Answer key

2.12.30 Grammar: GATE CSE 2006 | Question: 85



The grammar

- $S \rightarrow AC \mid CB$
- $C \rightarrow aCb \mid \epsilon$
- $A \rightarrow aA \mid a$
- $B \rightarrow Bb \mid b$

generates the language $L = \{a^i b^j \mid i \neq j\}$. In this grammar what is the length of the derivation (number of steps starting from S) to generate the string $a^l b^m$ with $l \neq m$

- A. $\max(l, m) + 2$ B. $l + m + 2$ C. $l + m + 3$ D. $\max(l, m) + 3$

gatecse-2006 compiler-design grammar normal

Answer key

2.12.31 Grammar: GATE CSE 2007 | Question: 52



Consider the grammar with non-terminals $N = \{S, C, S_1\}$, terminals $T = \{a, b, i, t, e\}$, with S as the start symbol, and the following set of rules:

$S \rightarrow iCtSS_1 \mid a$

$S_1 \rightarrow eS \mid \epsilon$

$C \rightarrow b$

The grammar is NOT LL(1) because:

- A. it is left recursive B. it is right recursive
C. it is ambiguous D. it is not context-free

gatecse-2007 compiler-design grammar normal

Answer key

2.12.32 Grammar: GATE CSE 2007 | Question: 53



Consider the following two statements:

- P: Every regular grammar is LL(1)
- Q: Every regular set has a LR(1) grammar

Which of the following is **TRUE**?

- A. Both P and Q are true B. P is true and Q is false
C. P is false and Q is true D. Both P and Q are false

gatecse-2007 compiler-design grammar normal

Answer key

2.12.33 Grammar: GATE CSE 2007 | Question: 78



Consider the CFG with $\{S, A, B\}$ as the non-terminal alphabet, $\{a, b\}$ as the terminal alphabet, S as the start symbol and the following set of production rules:

$S \rightarrow aB$ $S \rightarrow bA$

$B \rightarrow b$ $A \rightarrow a$

$B \rightarrow bS$ $A \rightarrow aS$

$B \rightarrow aBB$ $A \rightarrow bAA$

Which of the following strings is generated by the grammar?

- A. $aaaabb$ B. $aabbbb$ C. $aabbab$ D. $abbbba$

gatecse-2007 compiler-design grammar normal

Answer key

2.12.34 Grammar: GATE CSE 2007 | Question: 79



Consider the CFG with $\{S, A, B\}$ as the non-terminal alphabet, $\{a, b\}$ as the terminal alphabet, S as the start symbol and the following set of production rules:

$$\begin{aligned} S &\rightarrow aB & S &\rightarrow bA \\ B &\rightarrow b & A &\rightarrow a \\ B &\rightarrow bS & A &\rightarrow aS \\ B &\rightarrow aBB & A &\rightarrow bAA \end{aligned}$$

For the string $aabbab$, how many derivation trees are there?

- A. 1 B. 2 C. 3 D. 4

gatecse-2007 compiler-design grammar normal

Answer key

2.12.35 Grammar: GATE CSE 2008 | Question: 50



Which of the following statements are true?

- I. Every left-recursive grammar can be converted to a right-recursive grammar and vice-versa
- II. All ϵ -productions can be removed from any context-free grammar by suitable transformations
- III. The language generated by a context-free grammar all of whose productions are of the form $X \rightarrow w$ or $X \rightarrow wY$ (where, w is a string of terminals and Y is a non-terminal), is always regular
- IV. The derivation trees of strings generated by a context-free grammar in Chomsky Normal Form are always binary trees

- A. I, II, III and IV B. II, III and IV only
C. I, III and IV only D. I, II and IV only

gatecse-2008 normal compiler-design grammar

Answer key

2.12.36 Grammar: GATE CSE 2010 | Question: 38



The grammar $S \rightarrow aSa \mid bS \mid c$ is

- A. LL(1) but not LR(1) B. LR(1) but not LL(1)
C. Both LL(1) and LR(1) D. Neither LL(1) nor LR(1)

gatecse-2010 compiler-design grammar normal

Answer key

2.12.37 Grammar: GATE CSE 2016 | Set 2 | Question: 45



Which one of the following grammars is free from left recursion?

- | | | | |
|---------------------------|--------------------------------------|--|--------------------------------------|
| A. $S \rightarrow AB$ | B. $S \rightarrow Ab \mid Bb \mid c$ | C. $S \rightarrow Aa \mid B$ | D. $S \rightarrow Aa \mid Bb \mid c$ |
| $A \rightarrow Aa \mid b$ | $A \rightarrow Bd \mid \epsilon$ | $A \rightarrow Bb \mid Sc \mid \epsilon$ | $A \rightarrow Bd \mid \epsilon$ |
| $B \rightarrow c$ | $B \rightarrow e$ | $B \rightarrow d$ | $B \rightarrow Ae \mid \epsilon$ |

gatecse-2016-set2 compiler-design grammar easy

Answer key

2.12.38 Grammar: GATE CSE 2016 | Set 2 | Question: 46



A student wrote two context-free grammars G1 and G2 for generating a single C-like array declaration. The dimension of the array is at least one. For example,

```
int a[10][3];
```

The grammars use D as the start symbol, and use six terminal symbols `int ; id [ ] num.`

Grammar G1	Grammar G2
$D \rightarrow \text{int } L;$	$D \rightarrow \text{int } L;$
$L \rightarrow \text{id } [E$	$L \rightarrow \text{id } E$
$E \rightarrow \text{num }]$	$E \rightarrow E [\text{num}]$
$E \rightarrow \text{num }] [E$	$E \rightarrow [\text{num}]$

Which of the grammars correctly generate the declaration mentioned above?

- A. Both G1 and G2 B. Only G1 C. Only G2 D. Neither G1 nor G2

gatecse-2016-set2 compiler-design grammar normal

Answer key

2.12.39 Grammar: GATE CSE 2017 | Set 2 | Question: 32



Consider the following expression grammar G :

- $E \rightarrow E - T \mid T$
- $T \rightarrow T + F \mid F$
- $F \rightarrow (E) \mid id$

Which of the following grammars is not left recursive, but is equivalent to G ?

- A. $E \rightarrow E - T \mid T$
 $T \rightarrow T + F \mid F$
 $F \rightarrow (E) \mid id$
- B. $E \rightarrow TE'$
 $E' \rightarrow -TE' \mid \epsilon$
 $T \rightarrow T + F \mid F$
 $F \rightarrow (E) \mid id$
- C. $E \rightarrow TX$
 $X \rightarrow -TX \mid \epsilon$
 $T \rightarrow FY$
 $Y \rightarrow +FY \mid \epsilon$
 $F \rightarrow (E) \mid id$
- D. $E \rightarrow TX \mid (TX)$
 $X \rightarrow -TX \mid +TX \mid \epsilon$
 $T \rightarrow id$

gatecse-2017-set2 grammar

Answer key

2.12.40 Grammar: GATE CSE 2019 | Question: 43



Consider the augmented grammar given below:

- $S' \rightarrow S$
- $S \rightarrow \langle L \rangle \mid id$
- $L \rightarrow L, S \mid S$

Let $I_0 = \text{CLOSURE}(\{[S' \rightarrow \cdot S]\})$. The number of items in the set $\text{GOTO}(I_0, \langle \rangle)$ is _____

gatecse-2019 numerical-answers compiler-design grammar two-marks

Answer key

2.12.41 Grammar: GATE CSE 2021 | Set 1 | Question: 31



Consider the following context-free grammar where the set of terminals is $\{a, b, c, d, f\}$.

- $$\begin{aligned} S &\rightarrow daT \mid Rf \\ T &\rightarrow aS \mid baT \mid \epsilon \\ R &\rightarrow caTR \mid \epsilon \end{aligned}$$

The following is a partially-filled LL(1) parsing table.

	a	b	c	d	f	$\$$
S			[1]	$S \rightarrow daT$	[2]	
T	$T \rightarrow aS$	$T \rightarrow baT$	[3]		$T \rightarrow \epsilon$	[4]
R			$R \rightarrow caTR$		$R \rightarrow \epsilon$	

Which one of the following choices represents the correct combination for the numbered cells in the parsing table ("blank" denotes that the corresponding cell is empty)?

- A. [1] $S \rightarrow Rf$ [2] $S \rightarrow Rf$ [3] $T \rightarrow \epsilon$ [4] $T \rightarrow \epsilon$
 B. [1] blank [2] $S \rightarrow Rf$ [3] $T \rightarrow \epsilon$ [4] $T \rightarrow \epsilon$
 C. [1] $S \rightarrow Rf$ [2] blank [3] blank [4] $T \rightarrow \epsilon$
 D. [1] blank [2] $S \rightarrow Rf$ [3] blank [4] blank

gatecse-2021-set1 compiler-design grammar two-marks

Answer key

2.12.42 Grammar: GATE CSE 2024 | Set 1 | Question: 49

Let $G = (V, \Sigma, S, P)$ be a context-free grammar in Chomsky Normal Form with $\Sigma = \{a, b, c\}$ and V containing 10 variable symbols including the start symbol S . The string $w = a^{30}b^{30}c^{30}$ is derivable from S . The number of steps (application of rules) in the derivation $S \rightarrow^* w$ is _____.

gatecse-2024-set1 numerical-answers compiler-design grammar two-marks

Answer key

2.12.43 Grammar: GATE CSE 2024 | Set 2 | Question: 42

Consider a context-free grammar G with the following 3 rules.

$$S \rightarrow aS, S \rightarrow aSbS, S \rightarrow c$$

Let $w \in L(G)$. Let $n_a(w), n_b(w), n_c(w)$ denote the number of times a, b, c occur in w , respectively. Which of the following statements is/are TRUE?

- A. $n_a(w) > n_b(w)$ B. $n_a(w) > n_c(w) - 2$
 C. $n_c(w) = n_b(w) + 1$ D. $n_c(w) = n_b(w) * 2$

gatecse-2024-set2 compiler-design multiple-selects grammar two-marks

Answer key

2.12.44 Grammar: GATE CSE 2025 | Set 2 | Question: 41

Consider two grammars G_1 and G_2 with the production rules given below:

$$G_1 : S \rightarrow if\ E\ then\ S \mid if\ E\ then\ S\ else\ S \mid a \\ E \rightarrow b$$

$$G_2 : S \rightarrow if\ E\ then\ S \mid M \\ M \rightarrow if\ E\ then\ M\ else\ S \mid c \\ E \rightarrow b$$

where $if, then, else, a, b, c$ are the terminals.

Which of the following option(s) is/are CORRECT?

- A. G_1 is not $LL(1)$ and G_2 is $LL(1)$ B. G_1 is $LL(1)$ and G_2 is not $LL(1)$
 C. G_1 and G_2 are not $LL(1)$ D. G_1 and G_2 are ambiguous.

gatecse2025-set2 compiler-design grammar multiple-selects easy two-marks

Answer key

2.12.45 Grammar: GATE IT 2005 | Question: 83a



Consider the context-free grammar

$$\begin{aligned} E &\rightarrow E + E \\ E &\rightarrow (E * E) \\ E &\rightarrow id \end{aligned}$$

where E is the starting symbol, the set of terminals is $\{id, (, +,), *\}$, and the set of nonterminals is $\{E\}$.

Which of the following terminal strings has more than one parse tree when parsed according to the above grammar?

- A. $id + id + id + id$
- B. $id + (id * (id * id))$
- C. $(id * (id * id)) + id$
- D. $((id * id + id) * id)$

gateit-2005 compiler-design grammar parsing easy

Answer key

2.12.46 Grammar: GATE IT 2007 | Question: 9



Consider an ambiguous grammar G and its disambiguated version D . Let the language recognized by the two grammars be denoted by $L(G)$ and $L(D)$ respectively. Which one of the following is true?

- A. $L(D) \subset L(G)$
- B. $L(D) \supset L(G)$
- C. $L(D) = L(G)$
- D. $L(D)$ is empty

gateit-2007 compiler-design grammar normal

Answer key

2.12.47 Grammar: GATE IT 2008 | Question: 78



A CFG G is given with the following productions where S is the start symbol, A is a non-terminal and a and b are terminals.

- $S \rightarrow aS \mid A$
- $A \rightarrow aAb \mid bAa \mid \epsilon$

Which of the following strings is generated by the grammar above?

- A. $aabbaba$
- B. $aabaaba$
- C. $abababb$
- D. $aabbaab$

gateit-2008 parsing normal grammar

Answer key

2.13 Intermediate Code (11)

Practice Test: Test 1 (9Q)

2.13.1 Intermediate Code: GATE CSE 1988 | Question: 2xvii



Construct a DAG for the following set of quadruples:

- $E := A + B$
- $F := E - C$
- $G := F * D$
- $H := A + B$
- $I := I - C$
- $J := I + G$

gate1988 descriptive compiler-design intermediate-code

Answer key

2.13.2 Intermediate Code: GATE CSE 1989 | Question: 4-v



Is the following code template for the if-then-else statement correct? if not, correct it.

- if expression then statement 1
else statement 2

Template:

Code for expression

- (*result in E , $E > O$ indicates true *)
- Branch on $E > O$ to $L1$
- Code for statement 1
- $L1$: Code for statement 2

descriptive gate1989 compiler-design intermediate-code

Answer key

2.13.3 Intermediate Code: GATE CSE 1992 | Question: 11b



Write 3 address intermediate code (quadruples) for the following boolean expression in the sequence as it would be generated by a compiler. Partial evaluation of boolean expressions is not permitted. Assume the usual rules of precedence of the operators.

$$(a + b) > (c + d) \text{ or } a > c \text{ and } b < d$$

gate1992 compiler-design syntax-directed-translation intermediate-code descriptive

Answer key

2.13.4 Intermediate Code: GATE CSE 1994 | Question: 1.12



Generation of intermediate code based on an abstract machine model is useful in compilers because

- A. it makes implementation of lexical analysis and syntax analysis easier
- B. syntax-directed translations can be written for intermediate code generation
- C. it enhances the portability of the front end of the compiler
- D. it is not possible to generate code for real machines directly from high level language programs

gate1994 compiler-design intermediate-code easy

Answer key

2.13.5 Intermediate Code: GATE CSE 2014 | Set 2 | Question: 34



For a C program accessing $X[i][j][k]$, the following intermediate code is generated by a compiler. Assume that the size of an **integer** is 32 bits and the size of a **character** is 8 bits.

```
t0 = i * 1024
t1 = j * 32
t2 = k * 4
t3 = t1 + t0
t4 = t3 + t2
t5 = X[t4]
```

Which one of the following statements about the source code for the C program is CORRECT?

- A. X is declared as "int $X[32][32][8]$ ".
- B. X is declared as "int $X[4][1024][32]$ ".
- C. X is declared as "char $X[4][32][8]$ ".
- D. X is declared as "char $X[32][16][2]$ ".

gatecse-2014-set2 compiler-design intermediate-code programming-in-c normal

Answer key

2.13.6 Intermediate Code: GATE CSE 2014 | Set 3 | Question: 17



One of the purposes of using intermediate code in compilers is to

- A. make parsing and semantic analysis simpler.
- B. improve error recovery and error reporting.
- C. increase the chances of reusing the machine-independent code optimizer in other compilers.
- D. improve the register allocation.

gatecse-2014-set3 compiler-design intermediate-code easy

Answer key

2.13.7 Intermediate Code: GATE CSE 2015 | Set 1 | Question: 8



For computer based on three-address instruction formats, each address field can be used to specify which of the following:

- (S1) A memory operand
- (S2) A processor register
- (S3) An implied accumulator register

- A. Either $S1$ or $S2$
- B. Either $S2$ or $S3$
- C. Only $S2$ and $S3$
- D. All of $S1$, $S2$ and $S3$

gatecse-2015-set1 compiler-design intermediate-code normal

Answer key

2.13.8 Intermediate Code: GATE CSE 2015 | Set 2 | Question: 29



Consider the intermediate code given below.

```
(1) i=1
(2) j=1
(3) t1 = 5 * i
(4) t2 = t1 + j
(5) t3 = 4 * t2
(6) t4 = t3
(7) a[t4] = -1
(8) j = j + 1
(9) if j <= 5 goto (3)
(10) i = i + 1
(11) if i < 5 goto (2)
```

The number of nodes and edges in control-flow-graph constructed for the above code, respectively, are

- A. 5 and 7
- B. 6 and 7
- C. 5 and 5
- D. 7 and 8

gatecse-2015-set2 compiler-design intermediate-code normal

Answer key

2.13.9 Intermediate Code: GATE CSE 2021 | Set 2 | Question: 13



In the context of compilers, which of the following is/are NOT an intermediate representation of the source program?

- A. Three address code
- B. Abstract Syntax Tree (AST)
- C. Control Flow Graph (CFG)
- D. Symbol table

gatecse-2021-set2 multiple-selects compiler-design easy intermediate-code one-mark

Answer key

2.13.10 Intermediate Code: GATE CSE 2024 | Set 1 | Question: 29



Consider the following pseudo-code.

```
L 1: t1 = -1
```

```

L 2: t2 = 0
L 3: t3 = 0
L 4: t4 = 4 * t3
L 5: t5 = 4 * t2
L 6: t6 = t5 * M
L 7: t7 = t4 + t6
L 8: t8 = a[t7]
L 9: if t8 <= max goto L11
L 10: t1 = t8
L 11: t3 = t3 + 1
L 12: if t3 < M goto L4
L 13: t2 = t2 + 1
L 14: if t2 < N goto L3
L 15: max = t1

```

Which one of the following options CORRECTLY specifies the number of basic blocks and the number of instructions in the largest basic block, respectively?

- A. 6 and 6 B. 6 and 7 C. 7 and 7 D. 7 and 6

gatecse-2024-set1 compiler-design intermediate-code two-marks

Answer key

2.13.11 Intermediate Code: GATE CSE 2024 | Set 2 | Question: 33



Consider the following expression: $x[i] = (p + r) * -s[i] + u/w$. The following sequence shows the list of triples representing the given expression, with entries missing for triples (1), (3), and (6).

(0)	+	p	r
(1)			
(2)	uminus	(1)	
(3)			
(4)	/	u	w
(5)	+	(3)	(4)
(6)			
(7)	=	(6)	(5)

Which one of the following options fills in the missing entries CORRECTLY?

- A. (1) = [] s i (3) * (0)(2) (6) [] = x i
 B. (1) [] = s i (3) - (0)(2) (6) = [] x (5)
 C. (1) = [] s i (3) * (0)(2) (6) [] = x (5)
 D. (1) [] = s i (3) - (0)(2) (6) = [] x i

gatecse-2024-set2 compiler-design intermediate-code two-marks

Answer key

2.14

LR Parser (20)

Practice Tests: [Test 1 \(15Q\)](#) [Test 2 \(15Q\)](#) [Test 3 \(7Q\)](#)

2.14.1 LR Parser: GATE CSE 1992 | Question: 02,xiv



Consider the SLR(1) and LALR (1) parsing tables for a context free grammar. Which of the following statement is/are true?

- A. The *goto* part of both tables may be different.
- B. The *shift* entries are identical in both the tables.
- C. The *reduce* entries in the tables may be different.
- D. The *error* entries in tables may be different

gate1992 compiler-design normal parsing multiple-selects lr-parser

Answer key

2.14.2 LR Parser: GATE CSE 1998 | Question: 1.26



Which of the following statements is true?

- A. SLR parser is more powerful than LALR
- B. LALR parser is more powerful than Canonical LR parser
- C. Canonical LR parser is more powerful than LALR parser
- D. The parsers SLR, Canonical CR, and LALR have the same power

gate1998 compiler-design parsing normal lr-parser

Answer key

2.14.3 LR Parser: GATE CSE 2003 | Question: 17



Assume that the SLR parser for a grammar G has n_1 states and the LALR parser for G has n_2 states. The relationship between n_1 and n_2 is

- A. n_1 is necessarily less than n_2
- B. n_1 is necessarily equal to n_2
- C. n_1 is necessarily greater than n_2
- D. None of the above

gatecse-2003 compiler-design parsing easy lr-parser

Answer key

2.14.4 LR Parser: GATE CSE 2005 | Question: 60



Consider the grammar:

$$S \rightarrow (S) \mid a$$

Let the number of states in SLR (1), LR(1) and LALR(1) parsers for the grammar be n_1 , n_2 and n_3 respectively. The following relationship holds good:

- A. $n_1 < n_2 < n_3$
- B. $n_1 = n_3 < n_2$
- C. $n_1 = n_2 = n_3$
- D. $n_1 \geq n_3 \geq n_2$

gatecse-2005 compiler-design parsing normal lr-parser

Answer key

2.14.5 LR Parser: GATE CSE 2006 | Question: 7



Consider the following grammar

- $S \rightarrow S * E$
- $S \rightarrow E$
- $E \rightarrow F + E$
- $E \rightarrow F$
- $F \rightarrow id$

Consider the following LR(0) items corresponding to the grammar above

- i. $S \rightarrow S*.E$

- ii. $E \rightarrow F. + E$
 iii. $E \rightarrow F + . E$

Given the items above, which two of them will appear in the same set in the canonical sets-of-items for the grammar?

- A. i and ii B. ii and iii C. i and iii D. None of the above

gatecse-2006 compiler-design parsing normal lr-parser

Answer key

2.14.6 LR Parser: GATE CSE 2008 | Question: 55



An LALR(1) parser for a grammar G can have shift-reduce (S-R) conflicts if and only if

- A. The SLR(1) parser for G has S-R conflicts
 B. The LR(1) parser for G has S-R conflicts
 C. The LR(0) parser for G has S-R conflicts
 D. The LALR(1) parser for G has reduce-reduce conflicts

gatecse-2008 compiler-design parsing normal lr-parser

Answer key

2.14.7 LR Parser: GATE CSE 2013 | Question: 40



Consider the following two sets of LR(1) items of an LR(1) grammar.

$$\begin{array}{l|l} X \rightarrow c. X, c / d & X \rightarrow c. X, \$ \\ X \rightarrow . cX, c / d & X \rightarrow . cX, \$ \\ X \rightarrow . d, c / d & X \rightarrow . d, \$ \end{array}$$

Which of the following statements related to merging of the two sets in the corresponding LALR parser is/are FALSE?

1. Cannot be merged since look aheads are different.
 2. Can be merged but will result in S-R conflict.
 3. Can be merged but will result in R-R conflict.
 4. Cannot be merged since goto on c will lead to two different sets.
- A. 1 only B. 2 only
 C. 1 and 4 only D. 1, 2, 3 and 4

gatecse-2013 compiler-design parsing normal lr-parser

Answer key

2.14.8 LR Parser: GATE CSE 2013 | Question: 9



What is the maximum number of reduce moves that can be taken by a bottom-up parser for a grammar with no epsilon and unit-production (i.e., of type $A \rightarrow \epsilon$ and $A \rightarrow a$) to parse a string with n tokens?

- A. $n/2$ B. $n - 1$ C. $2n - 1$ D. 2^n

gatecse-2013 compiler-design parsing normal lr-parser

Answer key

2.14.9 LR Parser: GATE CSE 2014 | Set 1 | Question: 34



A canonical set of items is given below

- $S \rightarrow L. > R$
- $Q \rightarrow R.$

On input symbol $<$ the set has

- A. a shift-reduce conflict and a reduce-reduce conflict.
- B. a shift-reduce conflict but not a reduce-reduce conflict.
- C. a reduce-reduce conflict but not a shift-reduce conflict.
- D. neither a shift-reduce nor a reduce-reduce conflict.

gatecse-2014-set1 compiler-design parsing normal lr-parser

Answer key

2.14.10 LR Parser: GATE CSE 2015 | Set 3 | Question: 16



Among simple LR (SLR), canonical LR, and look-ahead LR (LALR), which of the following pairs identify the method that is very easy to implement and the method that is the most powerful, in that order?

- A. SLR, LALR
- B. Canonical LR, LALR
- C. SLR, canonical LR
- D. LALR, canonical LR

gatecse-2015-set3 compiler-design parsing normal lr-parser

Answer key

2.14.11 LR Parser: GATE CSE 2017 | Set 2 | Question: 6



Which of the following statements about parser is/are CORRECT?

- I. Canonical LR is more powerful than SLR
- II. SLR is more powerful than LALR
- III. SLR is more powerful than Canonical LR

- A. I only
- B. II only
- C. III only
- D. II and III only

gatecse-2017-set2 compiler-design parsing lr-parser

Answer key

2.14.12 LR Parser: GATE CSE 2019 | Question: 3



Which one of the following kinds of derivation is used by LR parsers?

- A. Leftmost
- B. Leftmost in reverse
- C. Rightmost
- D. Rightmost in reverse

gatecse-2019 compiler-design parsing one-mark lr-parser

Answer key

2.14.13 LR Parser: GATE CSE 2020 | Question: 24



Consider the following grammar.

- $S \rightarrow aSB \mid d$
- $B \rightarrow b$

The number of reduction steps taken by a bottom-up parser while accepting the string $aaadbbs$ is _____.

gatecse-2020 numerical-answers compiler-design lr-parser one-mark

Answer key

2.14.14 LR Parser: GATE CSE 2021 | Set 1 | Question: 5



Consider the following statements.

- S_1 : Every SLR(1) grammar is unambiguous but there are certain unambiguous grammars that are not SLR(1).
- S_2 : For any context-free grammar, there is a parser that takes at most $O(n^3)$ time to parse a string of length n .

Which one of the following options is correct?

- A. S_1 is true and S_2 is false
 C. S_1 is true and S_2 is true

- B. S_1 is false and S_2 is true
 D. S_1 is false and S_2 is false

gatecse-2021-set1 compiler-design lr-parser one-mark

Answer key

2.14.15 LR Parser: GATE CSE 2021 | Set 2 | Question: 51



Consider the following augmented grammar with $\{\#, @, <, >, a, b, c\}$ as the set of terminals.

$$\begin{aligned} S' &\rightarrow S \\ S &\rightarrow S\#cS \\ S &\rightarrow SS \\ S &\rightarrow S@ \\ S &\rightarrow <S> \\ S &\rightarrow a \\ S &\rightarrow b \\ S &\rightarrow c \end{aligned}$$

Let $I_0 = \text{CLOSURE}(\{S' \rightarrow \bullet S\})$. The number of items in the set $\text{GOTO}(\text{GOTO}(I_0, <), <)$ is _____

gatecse-2021-set2 compiler-design lr-parser numerical-answers two-marks

Answer key

2.14.16 LR Parser: GATE CSE 2022 | Question: 19



Consider the augmented grammar with $\{+, *, (,), \text{id}\}$ as the set of terminals.

- $S' \rightarrow S$
- $S \rightarrow S + R \mid R$
- $R \rightarrow R * P \mid P$
- $P \rightarrow (S) \mid \text{id}$

If I_0 is the set of two $LR(0)$ items $\{[S' \rightarrow S.], [S \rightarrow S. + R]\}$, then $\text{goto}(\text{closure}(I_0), +)$ contains exactly _____ items.

gatecse-2022 numerical-answers compiler-design parsing lr-parser one-mark

Answer key

2.14.17 LR Parser: GATE CSE 2022 | Question: 3



Which one of the following statements is TRUE?

- A. The $LALR(1)$ parser for a grammar G cannot have reduce-reduce conflict if the $LR(1)$ parser for G does not have reduce-reduce conflict.
 B. Symbol table is accessed only during the lexical analysis phase.
 C. Data flow analysis is necessary for run-time memory management.
 D. $LR(1)$ parsing is sufficient for deterministic context-free languages.

gatecse-2022 compiler-design parsing one-mark lr-parser

Answer key

2.14.18 LR Parser: GATE CSE 2024 | Set 2 | Question: 55



Consider the following augmented grammar, which is to be parsed with a SLR parser. The set of terminals is $\{a, b, c, d, \#, @\}$

$$\begin{aligned}
 S' &\rightarrow S \\
 S &\rightarrow SS|Aa|bAc|Bc|bBa \\
 A &\rightarrow d\# \\
 B &\rightarrow @
 \end{aligned}$$

Let $I_0 = \text{CLOSURE}(\{S' \rightarrow \bullet S\})$. The number of items in the set $\text{GOTO}(I_0, S)$ is _____.

gatecse-2024-set2 numerical-answers compiler-design lr-parser two-marks

Answer key

2.14.19 LR Parser: GATE CSE 2025 | Set 2 | Question: 30



Given a Context-Free Grammar G as follows:

$$\begin{aligned}
 S &\rightarrow Aa|bAc|dc | bda \\
 A &\rightarrow d
 \end{aligned}$$

Which ONE of the following statements is TRUE?

- A. G is neither $\text{LALR}(1)$ nor $\text{SLR}(1)$
- B. G is $\text{CLR}(1)$, not $\text{LALR}(1)$
- C. G is $\text{LALR}(1)$, not $\text{SLR}(1)$
- D. G is $\text{LALR}(1)$, also $\text{SLR}(1)$

gatecse2025-set2 compiler-design context-free-grammar parsing lr-parser two-marks

Answer key

2.14.20 LR Parser: GATE CSE 2026 | Set 2 | Question: 31



Consider the canonical $LR(0)$ parsing of the grammar below using terminals $\{a, b, c\}$ and non-terminals $\{A, B, C, S\}$ with S as the start symbol.

$$\begin{aligned}
 S &\rightarrow ACB \\
 A &\rightarrow aA | \epsilon \\
 C &\rightarrow cC | \epsilon \\
 B &\rightarrow bB | b
 \end{aligned}$$

Which one of the following options gives the number of shift-reduce conflicts that will occur in the $LR(0)$ ACTION table?

- A. 2
- B. 3
- C. 4
- D. 5

gatecse-2026-set2 compiler-design lr-parser two-marks

Answer key

2.15

Lexical Analysis (6)

Practice Tests: [Test 1 \(10Q\)](#) [Weekly Quiz 1 \(10Q\)](#)

2.15.1 Lexical Analysis: GATE CSE 1987 | Question: 1-xvii



Using longer identifiers in a program will necessarily lead to:

- A. Somewhat slower compilation
- B. A program that is easier to understand

C. An incorrect program

D. None of the above

gate1987 compiler-design lexical-analysis

Answer key

2.15.2 Lexical Analysis: GATE CSE 2000 | Question: 1.18, ISRO2015-25



The number of tokens in the following C statement is

```
printf("i=%d, &i=%x", i, &i);
```

A. 3

B. 26

C. 10

D. 21

gatecse-2000 compiler-design lexical-analysis easy isro2015

Answer key

2.15.3 Lexical Analysis: GATE CSE 2010 | Question: 13



Which data structure in a compiler is used for managing information about variables and their attributes?

A. Abstract syntax tree

B. Symbol table

C. Semantic stack

D. Parse table

gatecse-2010 compiler-design lexical-analysis easy

Answer key

2.15.4 Lexical Analysis: GATE CSE 2011 | Question: 1



In a compiler, keywords of a language are recognized during

A. parsing of the program

B. the code generation

C. the lexical analysis of the program

D. dataflow analysis

gatecse-2011 compiler-design lexical-analysis easy

Answer key

2.15.5 Lexical Analysis: GATE CSE 2011 | Question: 19



The lexical analysis for a modern computer language such as Java needs the power of which one of the following machine models in a necessary and sufficient sense?

A. Finite state automata

B. Deterministic pushdown automata

C. Non-deterministic pushdown automata

D. Turing machine

gatecse-2011 compiler-design lexical-analysis easy

Answer key

2.15.6 Lexical Analysis: GATE CSE 2018 | Question: 37



A lexical analyzer uses the following patterns to recognize three tokens T_1 , T_2 , and T_3 over the alphabet $\{a, b, c\}$.

- $T_1 : a?(b | c)^*a$
- $T_2 : b?(a | c)^*b$
- $T_3 : c?(b | a)^*c$

Note that ' $x?$ ' means 0 or 1 occurrence of the symbol x . Note also that the analyzer outputs the token that matches the longest possible prefix.

If the string *baacabc* is processed by the analyzer, which one of the following is the sequence of tokens it outputs?

A. $T_1T_2T_3$

B. $T_1T_1T_3$

C. $T_2T_1T_3$

D. T_3T_3

gatecse-2018 compiler-design lexical-analysis normal two-marks

Answer key

2.16.1 Linker: GATE CSE 1991 | Question: 03,ix



A “link editor” is a program that:

- A. matches the parameters of the macro-definition with locations of the parameters of the macro call
- B. matches external names of one program with their location in other programs
- C. matches the parameters of subroutine definition with the location of parameters of subroutine call.
- D. acts as a link between text editor and the user
- E. acts as a link between compiler and the user program

gate1991 compiler-design normal linker multiple-selects

Answer key

2.16.2 Linker: GATE CSE 2003 | Question: 76



Which of the following is NOT an advantage of using shared, dynamically linked libraries as opposed to using statically linked libraries?

- A. Smaller sizes of executable files
- B. Lesser overall page fault rate in the system
- C. Faster program startup
- D. Existing programs need not be re-linked to take advantage of newer versions of libraries

gatecse-2003 compiler-design runtime-environment linker easy

Answer key

2.16.3 Linker: GATE CSE 2004 | Question: 9



Consider a program P that consists of two source modules M_1 and M_2 contained in two different files. If M_1 contains a reference to a function defined in M_2 the reference will be resolved at

- A. Edit time
- B. Compile time
- C. Link time
- D. Load time

gatecse-2004 compiler-design easy linker

Answer key

2.17

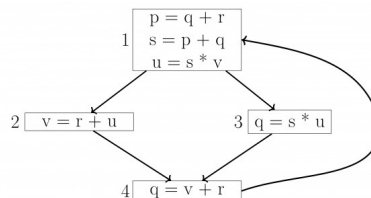
Live Variable Analysis (3)

2.17.1 Live Variable Analysis: GATE CSE 2015 | Set 1 | Question: 50



A variable x is said to be live at a statement S_i in a program if the following three conditions hold simultaneously:

- i. There exists a statement S_j that uses x
- ii. There is a path from S_i to S_j in the flow graph corresponding to the program
- iii. The path has no intervening assignment to x including at S_i and S_j



The variables which are live both at the statement in basic block 2 and at the statement in basic block 3 of the above control flow graph are

- A. p, s, u
- B. r, s, u
- C. r, u
- D. q, v

gatecse-2015-set1 compiler-design live-variable-analysis normal

Answer key

2.17.2 Live Variable Analysis: GATE CSE 2021 | Set 2 | Question: 38



For a statement S in a program, in the context of liveness analysis, the following sets are defined:

$USE(S)$: the set of variables used in S

$IN(S)$: the set of variables that are live at the entry of S

$OUT(S)$: the set of variables that are live at the exit of S

Consider a basic block that consists of two statements, S_1 followed by S_2 . Which one of the following statements is correct?

- A. $OUT(S_1) = IN(S_2)$
- B. $OUT(S_1) = IN(S_1) \cup USE(S_1)$
- C. $OUT(S_1) = IN(S_2) \cup OUT(S_2)$
- D. $OUT(S_1) = USE(S_1) \cup IN(S_2)$

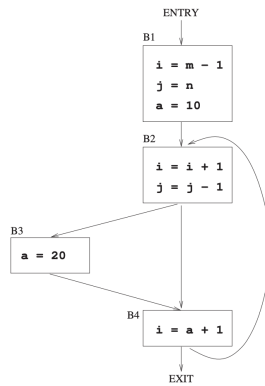
gatecse-2021-set2 code-optimization live-variable-analysis compiler-design two-marks

Answer key

2.17.3 Live Variable Analysis: GATE CSE 2023 | Question: 27



Consider the control flow graph shown.



Which one of the following choices correctly lists the set of *live* variables at the exit point of each basic block?

- A. B1: $\{ \}$, B2: $\{a\}$, B3: $\{a\}$, B4: $\{a\}$
- B. B1: $\{i, j\}$, B2: $\{a\}$, B3: $\{a\}$, B4: $\{i\}$
- C. B1: $\{a, i, j\}$, B2: $\{a, i, j\}$, B3: $\{a, i\}$, B4: $\{a\}$
- D. B1: $\{a, i, j\}$, B2: $\{a, j\}$, B3: $\{a, j\}$, B4: $\{a, i, j\}$

gatecse-2023 compiler-design live-variable-analysis two-marks

Answer key

2.18

LI Parser (2)

2.18.1 LI Parser: GATE CSE 2024 | Set 2 | Question: 30



Consider the following context-free grammar where the start symbol is S and the set of terminals is $\{a, b, c, d\}$.

$$S \rightarrow AaAb \mid BbBa$$

$$A \rightarrow cS \mid \epsilon$$

$$B \rightarrow dS \mid \epsilon$$

The following is a partially-filled $LL(1)$ parsing table.

	a	b	c	d	$\$$
S	$S \rightarrow AaAb$	$S \rightarrow BbBa$	(1)	(2)	
A	$A \rightarrow \epsilon$	(3)	$A \rightarrow cS$		
B	(4)	$B \rightarrow \epsilon$		$B \rightarrow dS$	

Which one of the following options represents the CORRECT combination for the numbered cells in the parsing table?

Note: In the options, "blank" denotes that the corresponding cell is empty.

- A. (1) $S \rightarrow AaAb$ (2) $S \rightarrow BbBa$ (3) $A \rightarrow \epsilon$ (4) $B \rightarrow \epsilon$
- B. (1) $S \rightarrow BbBa$ (2) $S \rightarrow AaAb$ (3) $A \rightarrow \epsilon$ (4) $B \rightarrow \epsilon$
- C. (1) $S \rightarrow AaAb$ (2) $S \rightarrow BbBa$ (3) blank (4) blank
- D. (1) $S \rightarrow BbBa$ (2) $S \rightarrow AaAb$ (3) blank (4) blank

gatecse-2024-set2 compiler-design ll-parser parsing two-marks

Answer key

2.18.2 LI Parser: GATE CSE 2026 | Set 1 | Question: 18



Which of the following statements is/are true?

- A. LL(1) parser uses backtracking
- B. For a grammar to be LL(1), it must be left-recursive
- C. For a grammar to be LL(1), it must be left-factored
- D. The LL(1) parsers are more powerful than the SLR parsers

gatecse-2026-set1 compiler-design parsing ll-parser multiple-selects one-mark

Answer key

2.19 Macros (4)

2.19.1 Macros: GATE CSE 1992 | Question: 01,vii



Macro expansion is done in pass one instead of pass two in a two pass macro assembler because

gate1992 compiler-design macros easy fill-in-the-blanks

Answer key

2.19.2 Macros: GATE CSE 1995 | Question: 1.11



What are x and y in the following macro definition?

```
macro Add x, y
    Load y
    Mul x
    Store y
end macro
```

- A. Variables
- B. Identifiers
- C. Actual parameters
- D. Formal parameters

gate1995 compiler-design macros easy

Answer key

2.19.3 Macros: GATE CSE 1996 | Question: 2.16



Which of the following macros can put a macro assembler into an infinite loop?

- i.

```
.MACRO M1, X
    .IF EQ, X ;if X=0 then
    M1 X + 1
```

```

.ENDC
.IF NE, X ;if X ≠ 0 then
.WORD X ;address (X) is stored here
.ENDC
.ENDM

```

ii.

```

.MACRO M2, X
.IF EQ, X
M2 X
.ENDC
.IF NE, X
.WORD X + 1
.ENDC
.ENDM

```

- A. (ii) only B. (i) only C. both (i) and (ii) D. None of the above

gate1996 compiler-design macros normal

Answer key

2.19.4 Macros: GATE CSE 1997 | Question: 1.9



The conditional expansion facility of macro processor is provided to

- A. test a condition during the execution of the expanded program
- B. to expand certain model statements depending upon the value of a condition during the execution of the expanded program
- C. to implement recursion
- D. to expand certain model statements depending upon the value of a condition during the process of macro expansion

gate1997 compiler-design macros easy

Answer key

2.20

Operator Precedence (9)

Practice Test: Test 1 (7Q)

2.20.1 Operator Precedence: GATE CSE 1991 | Question: 10a



Consider the following grammar for arithmetic expressions using binary operators — and / which are not associative

- $E \rightarrow E - T \mid T$
- $T \rightarrow T / F \mid F$
- $F \rightarrow (E) \mid id$

(E is the start symbol)

Is the grammar unambiguous? Is so, what is the relative precedence between — and /? If not, give an unambiguous grammar that gives / precedence over —.

gate1991 grammar compiler-design normal descriptive ambiguous-grammar operator-precedence

Answer key

2.20.2 Operator Precedence: GATE CSE 1997 | Question: 1.6



In the following grammar

- $X ::= X \oplus Y \mid Y$
- $Y ::= Z * Y \mid Z$
- $Z ::= id$

Which of the following is true?

- A. ' $\oplus$ ' is left associative while ' $*$ ' is right associative
 B. Both ' $\oplus$ ' and ' $*$ ' are left associative
 C. ' $\oplus$ ' is right associative while ' $*$ ' is left associative
 D. None of the above

gate1997 compiler-design grammar normal operator-precedence ambiguous-grammar

Answer key

2.20.3 Operator Precedence: GATE CSE 2000 | Question: 2.21, ISRO2015-24

Given the following expression grammar:

$$E \rightarrow E * F \mid F + E \mid F$$

$$F \rightarrow F - F \mid id$$

Which of the following is true?

- A. $*$ has higher precedence than $+$
 B. $-$ has higher precedence than $*$
 C. $+$ and $-$ have same precedence
 D. $+$ has higher precedence than $*$

gatecse-2000 operator-precedence normal compiler-design isro2015 ambiguous-grammar

Answer key

2.20.4 Operator Precedence: GATE CSE 2002 | Question: 22

- A. Construct all the parse trees corresponding to $i + j * k$ for the grammar

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

- B. In this grammar, what is the precedence of the two operators $*$ and $+$?
 C. If only one parse tree is desired for any string in the same language, what changes are to be made so that the resulting LALR(1) grammar is unambiguous?

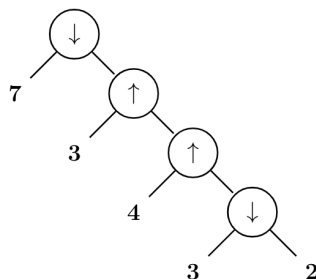
gatecse-2002 compiler-design parsing normal descriptive operator-precedence ambiguous-grammar

Answer key

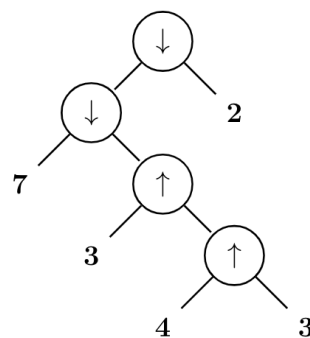
2.20.5 Operator Precedence: GATE CSE 2011 | Question: 27

Consider two binary operators ' $\uparrow$ ' and ' $\downarrow$ ' with the precedence of operator $\downarrow$ being lower than that of the operator $\uparrow$. Operator $\uparrow$ is right associative while operator $\downarrow$ is left associative. Which one of the following represents the parse tree for expression $(7 \downarrow 3 \uparrow 4 \uparrow 3 \downarrow 2)$

A.

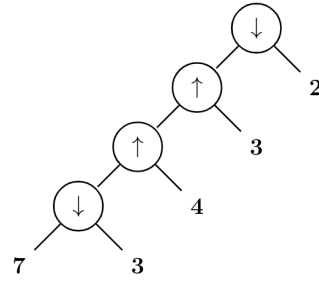
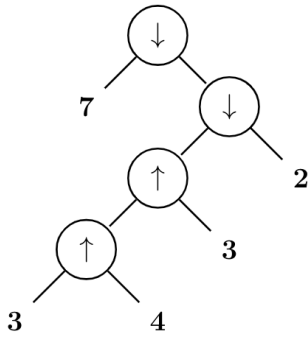


B.



C.

D.



gatecse-2011 compiler-design parsing normal operator-precedence ambiguous-grammar

Answer key

2.20.6 Operator Precedence: GATE CSE 2014 | Set 2 | Question: 17

Consider the grammar defined by the following production rules, with two operators $*$ and $+$

- $S \rightarrow T * P$
- $T \rightarrow U \mid T * U$
- $P \rightarrow Q + P \mid Q$
- $Q \rightarrow Id$
- $U \rightarrow Id$

Which one of the following is TRUE?

- A. $+$ is left associative, while $*$ is right associative
- B. $+$ is right associative, while $*$ is left associative
- C. Both $+$ and $*$ are right associative
- D. Both $+$ and $*$ are left associative

gatecse-2014-set2 compiler-design grammar normal operator-precedence ambiguous-grammar

Answer key

2.20.7 Operator Precedence: GATE CSE 2016 | Set 1 | Question: 45

The attribute of three arithmetic operators in some programming language are given below.

OPERATOR	PRECEDENCE	ASSOCIATIVITY	ARITY
$+$	High	Left	Binary
$-$	Medium	Right	Binary
$*$	Low	Left	Binary

The value of the expression $2 - 5 + 1 - 7 * 3$ in this language is _____.

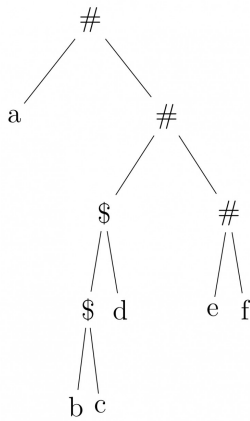
gatecse-2016-set1 compiler-design parsing normal numerical-answers operator-precedence

Answer key

2.20.8 Operator Precedence: GATE CSE 2018 | Question: 38

Consider the following parse tree for the expression $a\#b\$c\$d\#e\#f$, involving two binary operators $\$$ and $\#$.





Which one of the following is correct for the given parse tree?

- A. \$ has higher precedence and is left associative; # is right associative
- B. # has higher precedence and is left associative; \$ is right associative
- C. \$ has higher precedence and is left associative; # is left associative
- D. \$ has higher precedence and is right associative; # is left associative

gatecse-2018 compiler-design parsing normal two-marks operator-precedence ambiguous-grammar

Answer key

2.20.9 Operator Precedence: GATE CSE 2024 | Set 1 | Question: 23



Consider the operator precedence and associativity rules for the *integer* arithmetic operators given in the table below.

Operator	Precedence	Associativity
+	Highest	Left
−	High	Right
*	Medium	Right
/	Low	Right

The value of the expression $3 + 1 + 5 * 2/7 + 2 - 4 - 7 - 6/2$ as per the above rules is _____.

gatecse-2024-set1 numerical-answers compiler-design operator-precedence one-mark

Answer key

2.21 Parameter Passing (14)

Practice Test: Test 1 (7Q)

2.21.1 Parameter Passing: GATE CSE 1988 | Question: 2xv



What is printed by following program, assuming call-by reference method of passing parameters for all variables in the parameter list of procedure P?

```

program Main(inout, output);
var a, b:integer;
procedure P(x, y, z:integer);
begin
  y:=y+1
  z:=x+x
end P;
begin
  a:=2; b:=3;
  p(a+b, a, a);
  Write(a)
end.
  
```

Answer key

2.21.2 Parameter Passing: GATE CSE 1988 | Question: 8i



Consider the procedure declaration:

```
Procedure
P (k: integer)
```

where the parameter passing mechanism is call-by-value-result. Is it correct if the call, P (A[i]), where A is an array and i an integer, is implemented as below.

- | | |
|---|-------------------------------------|
| a. create a new local variable, say z; | b. assign to z, the value of A [i]; |
| c. execute the body of P using z for k; | d. set A [i] to z; |
- suggest a correct one. Explain your answer. If this is incorrect implementation,

Answer key

2.21.3 Parameter Passing: GATE CSE 1989 | Question: 3-viii



In which of the following case(s) is it possible to obtain different results for call-by-reference and call-by-name parameter passing?

- | | |
|---|--|
| A. Passing an expression as a parameter | B. Passing an array as a parameter |
| C. Passing a pointer as a parameter | D. Passing an array element as a parameter |

Answer key

2.21.4 Parameter Passing: GATE CSE 1990 | Question: 11a



What does the following program output?

```
program module (input, output);
var
  a:array [1..5] of integer;
  i, j: integer;
procedure unknown (var b: integer, var c: integer);
var
  i:integer;
begin
  for i := 1 to 5 do a[i] := i;
  b:= 0; c := 0
  for i := 1 to 5 do write (a[i]);
  writeln();
  a[3]:=11; a[1]:=11;
  for i:=1 to 5 do a [i] := sqr(a[i]);
  writeln(c,b); b := 5; c := 6;
end;
begin
  i:=1; j:=3; unknown (a[i], a[j]);
  for i:=1 to 5 do write (a[i]);
end;
```

Answer key

2.21.5 Parameter Passing: GATE CSE 1991 | Question: 03,x



Indicate all the true statements from the following:

- A. Recursive descent parsing cannot be used for grammar with left recursion.
 B. The intermediate form for representing expressions which is best suited for code optimization is the postfix form.
 C. A programming language not supporting either recursion or pointer type does not need the support of dynamic memory allocation.

- D. Although C does not support call-by-name parameter passing, the effect can be correctly simulated in C
E. No feature of Pascal typing violates strong typing in Pascal.

gate1991 compiler-design parameter-passing difficult multiple-selects

Answer key

2.21.6 Parameter Passing: GATE CSE 1991 | Question: 09a



Consider the following pseudo-code (all data items are of type integer):

```
procedure P(a, b, c);  
  a := 2;  
  c := a + b;  
end {P}  
  
begin  
  x := 1;  
  y := 5;  
  z := 100;  
  P(x, x*y, z);  
  Write ('x = ', x, 'z = ', z);  
end
```

Determine its output, if the parameters are passed to the Procedure **P** by

- value
- reference
- name

gate1991 compiler-design parameter-passing normal runtime-environment descriptive

Answer key

2.21.7 Parameter Passing: GATE CSE 1991 | Question: 09b



For the following code, indicate the output if

- static scope rules
- dynamic scope rules

are used

```
var a,b : integer;  
  
procedure P;  
  a := 5;  
  b := 10;  
end {P};  
  
procedure Q;  
  var a, b : integer;  
  P;  
end {Q};  
  
begin  
  a := 1;  
  b := 2;  
  Q;  
  Write ('a = ', a, 'b = ', b);  
end
```

gate1991 runtime-environment normal compiler-design parameter-passing descriptive

Answer key

2.21.8 Parameter Passing: GATE CSE 1993 | Question: 26



A stack is used to pass parameters to procedures in a procedure call.

- A. If a procedure *P* has two parameters as described in procedure definition:

```
procedure P (var x :integer; y: integer);
```

and if P is called by ; $P(a, b)$

State precisely in a sentence what is pushed on stack for parameters a and b

B. In the generated code for the body of procedure P , how will the addressing of formal parameters x and y differ?

gate1993 compiler-design parameter-passing runtime-environment normal descriptive

Answer key

2.21.9 Parameter Passing: GATE CSE 1995 | Question: 2.4



What is the value of X printed by the following program?

```
program COMPUTE (input, output);
var X:integer;
procedure FIND (X:real);
begin
  X:=sqrt(X);
end;
begin
  X:=2;
  FIND(X);
  writeln(X);
end.
```

A. 2

B. $\sqrt{2}$

C. Run time error

D. None of the above

gate1995 compiler-design parameter-passing runtime-environment easy

Answer key

2.21.10 Parameter Passing: GATE CSE 1999 | Question: 15



What will be the output of the following program assuming that parameter passing is

- call by value
- call by reference
- call by copy restore

```
procedure P{x, y, z};
begin
  y:=y+1;
  z:= x+x;
end;
begin
  a:= b:= 3;
  P(a+b, a, a);
  Print(a);
end
```

gate1999 parameter-passing normal runtime-environment descriptive

Answer key

2.21.11 Parameter Passing: GATE CSE 2003 | Question: 74



The following program fragment is written in a programming language that allows global variables and does not allow nested declarations of functions.

```
global int i=100, j=5;
void P(x) {
  int i=10;
  print(x+10);
  i=200;
  j=20;
  print (x);
}
main() {P(i+j);}
```

If the programming language uses dynamic scoping and call by name parameter passing mechanism, the values

printed by the above program are

- A. 115,220 B. 25,220 C. 25,15 D. 115,105

gatecse-2003 programming compiler-design parameter-passing runtime-environment normal

Answer key

2.21.12 Parameter Passing: GATE CSE 2004 | Question: 2,ISRO2017-54



Consider the following function

```
void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}
```

In order to exchange the values of two variables x and y .

- A. call $swap(x, y)$
B. call $swap(\&x, \&y)$
C. $swap(x, y)$ cannot be used as it does not return any value
D. $swap(x, y)$ cannot be used as the parameters are passed by value

gatecse-2004 compiler-design programming-in-c parameter-passing easy isro2017 runtime-environment

Answer key

2.21.13 Parameter Passing: GATE CSE 2016 | Set 1 | Question: 36



What will be the output of the following pseudo-code when parameters are passed by reference and dynamic scoping is assumed?

```
a = 3;
void n(x) { x = x * a; print (x); }
void m(y) { a = 1 ; a = y - a; n(a); print (a); }
void main () { m(a); }
```

- A. 6,2 B. 6,6 C. 4,2 D. 4,4

gatecse-2016-set1 parameter-passing normal

Answer key

2.21.14 Parameter Passing: GATE IT 2007 | Question: 33



Consider the program below in a hypothetical language which allows global variable and a choice of call by reference or call by value methods of parameter passing.

```
int i ;
program main ()
{
    int j = 60;
    i = 50;
    call f (i, j);
    print i, j;
}
procedure f (x, y)
{
    i = 100;
    x = 10;
    y = y + i ;
}
```

Which one of the following options represents the correct output of the program for the two parameter passing mechanisms?

- A. Call by value : $i = 70, j = 10$; Call by reference : $i = 60, j = 70$
B. Call by value : $i = 50, j = 60$; Call by reference : $i = 50, j = 70$

- C. Call by value : $i = 10, j = 70$; Call by reference : $i = 100, j = 60$
D. Call by value : $i = 100, j = 60$; Call by reference : $i = 10, j = 70$

gateit-2007 programming parameter-passing normal compiler-design runtime-environment

Answer key

2.22

Parsing (22)

 **Practice Tests:** Test 1 (15Q) Test 2 (15Q) Test 3 (7Q)

2.22.1 Parsing: GATE CSE 1987 | Question: 1-xiv

An operator precedence parser is a

- A. Bottom-up parser.
B. Top-down parser.
C. Back tracking parser.
D. None of the above.

gate1987 compiler-design parsing easy

Answer key

2.22.2 Parsing: GATE CSE 1989 | Question: 1-iii

Merging states with a common core may produce _____ conflicts and does not produce _____ conflicts in an LALR parser.

gate1989 descriptive compiler-design parsing

Answer key

2.22.3 Parsing: GATE CSE 1993 | Question: 25

A simple Pascal like language has only three statements.

- i. assignment statement e.g. $x := \text{expression}$
 - ii. loop construct e.g. for $i := \text{expression}$ to expression do statement
 - iii. sequencing e.g. begin statement ; ... ; statement end
- A. Write a context-free grammar (CFG) for statements in the above language. Assume that expression has already been defined. Do not use optional parenthesis and * operator in CFG.
- B. Show the parse tree for the following statements:

```
for j:=2 to 10 do
begin
    x:=expr1;
    y:=expr2;
end
```

gate1993 compiler-design parsing normal descriptive

Answer key 

2.22.4 Parsing: GATE CSE 1995 | Question: 8

Construct the LL(1) table for the following grammar.

1. $Expr \rightarrow _Expr$
2. $Expr \rightarrow (Expr)$
3. $Expr \rightarrow Var\ ExprTail$
4. $ExprTail \rightarrow _Expr$
5. $Expr \rightarrow \lambda$
6. $Var \rightarrow Id\ VarTail$
7. $VarTail \rightarrow (Expr)$
8. $VarTail \rightarrow \lambda$
9. $Goal \rightarrow Expr\$$

Answer key

2.22.5 Parsing: GATE CSE 1998 | Question: 1.27



Type checking is normally done during

- A. lexical analysis
 B. syntax analysis
 C. syntax directed translation
 D. code optimization

gate1998 compiler-design parsing easy

Answer key

2.22.6 Parsing: GATE CSE 1998 | Question: 22



- A. An identifier in a programming language consists of up to six letters and digits of which the first character must be a letter. Derive a regular expression for the identifier.
- B. Build an $LL(1)$ parsing table for the language defined by the $LL(1)$ grammar with productions
- Program $\rightarrow$ begin d semi X end
- $X \rightarrow d$ semi $X \mid sY$
- $Y \rightarrow$ semi $sY \mid \epsilon$

gate1998 compiler-design parsing descriptive

Answer key

2.22.7 Parsing: GATE CSE 1999 | Question: 1.17



Which of the following is the most powerful parsing method?

- A. LL (1) B. Canonical LR C. SLR D. LALR

gate1999 compiler-design parsing easy

Answer key

2.22.8 Parsing: GATE CSE 2000 | Question: 1.19, UGCNET-Dec2013-II: 30



Which of the following derivations does a top-down parser use while parsing an input string? The input is scanned from left to right.

- A. Leftmost derivation
 B. Leftmost derivation traced out in reverse
 C. Rightmost derivation
 D. Rightmost derivation traced out in reverse

gatecse-2000 compiler-design parsing normal ugcnetcse-dec2013-paper2

Answer key

2.22.9 Parsing: GATE CSE 2001 | Question: 16



Consider the following grammar with terminal alphabet $\Sigma = \{a, (,), +, *\}$ and start symbol E . The production rules of the grammar are:

- $E \rightarrow aA$
- $E \rightarrow (E)$
- $A \rightarrow +E$
- $A \rightarrow *E$
- $A \rightarrow \epsilon$

- a. Compute the FIRST and FOLLOW sets for E and A .
 b. Complete the $LL(1)$ parse table for the grammar.

Answer key

2.22.10 Parsing: GATE CSE 2003 | Question: 16



Which of the following suffices to convert an arbitrary CFG to an LL(1) grammar?

- A. Removing left recursion alone
- B. Factoring the grammar alone
- C. Removing left recursion and factoring the grammar
- D. None of the above

gatecse-2003 compiler-design parsing easy

Answer key

2.22.11 Parsing: GATE CSE 2005 | Question: 83a

**Statement for Linked Answer Questions 83a & 83b:**

Consider the following expression grammar. The semantic rules for expression evaluation are stated next to each grammar production.

$$\begin{array}{l|l}
 E \rightarrow \text{number} & E.val = \text{number}.val \\
 | E '+' E & E^{(1)}.val = E^{(2)}.val + E^{(3)}.val \\
 | E ' \times ' E & E^{(1)}.val = E^{(2)}.val \times E^{(3)}.val
 \end{array}$$

The above grammar and the semantic rules are fed to a *yacc* tool (which is an LALR(1) parser generator) for parsing and evaluating arithmetic expressions. Which one of the following is true about the action of *yacc* for the given grammar?

- A. It detects *recursion* and eliminates recursion
- B. It detects *reduce-reduce* conflict, and resolves
- C. It detects *shift-reduce* conflict, and resolves the conflict in favor of a *shift* over a *reduce* action
- D. It detects *shift-reduce* conflict, and resolves the conflict in favor of a *reduce* over a *shift* action

gatecse-2005 compiler-design parsing difficult

Answer key

2.22.12 Parsing: GATE CSE 2005 | Question: 83b



Consider the following expression grammar. The semantic rules for expression evaluation are stated next to each grammar production.

$$\begin{array}{l|l}
 E \rightarrow \text{number} & E.val = \text{number}.val \\
 | E '+' E & E^{(1)}.val = E^{(2)}.val + E^{(3)}.val \\
 | E ' \times ' E & E^{(1)}.val = E^{(2)}.val \times E^{(3)}.val
 \end{array}$$

Assume the conflicts of this question are resolved using yacc tool and an LALR(1) parser is generated for parsing arithmetic expressions as per the given grammar. Consider an expression $3 \times 2 + 1$. What precedence and associativity properties does the generated parser realize?

- A. Equal precedence and left associativity; expression is evaluated to 7
- B. Equal precedence and right associativity; expression is evaluated to 9
- C. Precedence of ' $\times$ ' is higher than that of '+', and both operators are left associative; expression is evaluated to 7
- D. Precedence of '+' is higher than that of ' $\times$ ', and both operators are left associative; expression is evaluated to 9

Answer key

2.22.13 Parsing: GATE CSE 2006 | Question: 58



Consider the following grammar:

- $S \rightarrow FR$
- $R \rightarrow *S \mid \varepsilon$
- $F \rightarrow id$

In the predictive parser table M of the grammar the entries $M[S, id]$ and $M[R, \$]$ respectively are

- A. $\{S \rightarrow FR\}$ and $\{R \rightarrow \varepsilon\}$
- B. $\{S \rightarrow FR\}$ and $\{\}$
- C. $\{S \rightarrow FR\}$ and $\{R \rightarrow *S\}$
- D. $\{F \rightarrow id\}$ and $\{R \rightarrow \varepsilon\}$

gatecse-2006 compiler-design parsing normal

Answer key

2.22.14 Parsing: GATE CSE 2007 | Question: 18



Which one of the following is a top-down parser?

- A. Recursive descent parser.
- B. Operator precedence parser.
- C. An LR(k) parser.
- D. An LALR(k) parser.

gatecse-2007 compiler-design parsing normal

Answer key

2.22.15 Parsing: GATE CSE 2008 | Question: 11



Which of the following describes a handle (as applicable to LR-parsing) appropriately?

- A. It is the position in a sentential form where the next shift or reduce operation will occur
- B. It is non-terminal whose production will be used for reduction in the next step
- C. It is a production that may be used for reduction in a future step along with a position in the sentential form where the next shift or reduce operation will occur
- D. It is the production p that will be used for reduction in the next step along with a position in the sentential form where the right hand side of the production may be found

gatecse-2008 compiler-design parsing normal

Answer key

2.22.16 Parsing: GATE CSE 2009 | Question: 42



Which of the following statements are TRUE?

- I. There exist parsing algorithms for some programming languages whose complexities are less than $\Theta(n^3)$
- II. A programming language which allows recursion can be implemented with static storage allocation.
- III. No L-attributed definition can be evaluated in the framework of bottom-up parsing.
- IV. Code improving transformations can be performed at both source language and intermediate code level.

- A. I and II
- B. I and IV
- C. III and IV
- D. I, III and IV

gatecse-2009 compiler-design parsing normal

Answer key

2.22.17 Parsing: GATE CSE 2012 | Question: 53



For the grammar below, a partial $LL(1)$ parsing table is also presented along with the grammar. Entries that

need to be filled are indicated as $E1$, $E2$, and $E3$. ϵ is the empty string, \$ indicates end of input, and, | separates alternate right hand sides of productions.

- $S \rightarrow aAbB \mid bAaB \mid \epsilon$
- $A \rightarrow S$
- $B \rightarrow S$

	a	b	\$
S	E1	E2	$S \rightarrow \epsilon$
A	$A \rightarrow S$	$A \rightarrow S$	error
B	$B \rightarrow S$	$B \rightarrow S$	E3

The appropriate entries for $E1$, $E2$, and $E3$ are

- A. $E1 : S \rightarrow aAbB, A \rightarrow S$
 $E2 : S \rightarrow bAaB, B \rightarrow S$
 $E3 : B \rightarrow S$
- B. $E1 : S \rightarrow aAbB, S \rightarrow \epsilon$
 $E2 : S \rightarrow bAaB, S \rightarrow \epsilon$
 $E3 : S \rightarrow \epsilon$
- C. $E1 : S \rightarrow aAbB, S \rightarrow \epsilon$
 $E2 : S \rightarrow bAaB, S \rightarrow \epsilon$
 $E3 : B \rightarrow S$
- D. $E1 : A \rightarrow S, S \rightarrow \epsilon$
 $E2 : B \rightarrow S, S \rightarrow \epsilon$
 $E3 : B \rightarrow S$

normal gatecse-2012 compiler-design parsing

Answer key

2.22.18 Parsing: GATE CSE 2015 | Set 3 | Question: 31



Consider the following grammar G

$S \rightarrow F \mid H$

$F \rightarrow p \mid c$

$H \rightarrow d \mid c$

Where S , F , and H are non-terminal symbols, p, d , and c are terminal symbols. Which of the following statement(s) is/are correct?

S1: LL(1) can parse all strings that are generated using grammar G

S2: LR(1) can parse all strings that are generated using grammar G

- A. Only S1 B. Only S2 C. Both S1 and S2 D. Neither S1 and S2

gatecse-2015-set3 compiler-design parsing normal

Answer key

2.22.19 Parsing: GATE CSE 2017 | Set 1 | Question: 43



Consider the following grammar:

- $\text{stmt} \rightarrow \text{if expr then expr else expr; stmt} \mid \emptyset$
- $\text{expr} \rightarrow \text{term relop term} \mid \text{term}$
- $\text{term} \rightarrow \text{id} \mid \text{number}$
- $\text{id} \rightarrow \mathbf{a} \mid \mathbf{b} \mid \mathbf{c}$
- $\text{number} \rightarrow [0 - 9]$

where **relop** is a relational operator (e.g., $<$, $>$, $\dots$), $\emptyset$ refers to the empty statement, and **if**, **then**, **else** are terminals.

Consider a program P following the above grammar containing ten **if** terminals. The number of control flow paths in P is _____. For example, the program

if e_1 **then** e_2 **else** e_3

has 2 control flow paths. $e_1 \rightarrow e_2$ and $e_1 \rightarrow e_3$.

gatecse-2017-set1 compiler-design parsing normal numerical-answers

Answer key

2.22.20 Parsing: GATE CSE 2024 | Set 1 | Question: 16



Which of the following is/are Bottom-Up Parser(s)?

- A. Shift-reduce Parser
- B. Predictive Parser
- C. LL(1) Parser
- D. LR Parser

gatecse-2024-set1 multiple-selects compiler-design parsing easy one-mark

Answer key

2.22.21 Parsing: GATE IT 2005 | Question: 83b



Consider the context-free grammar

- $E \rightarrow E + E$
- $E \rightarrow (E * E)$
- $E \rightarrow id$

where E is the starting symbol, the set of terminals is $\{id, (, +,), *\}$, and the set of non-terminals is $\{E\}$. For the terminal string $id + id + id + id$, how many parse trees are possible?

- A. 5
- B. 4
- C. 3
- D. 2

gateit-2005 compiler-design parsing normal

Answer key

2.22.22 Parsing: GATE IT 2008 | Question: 79



A CFG G is given with the following productions where S is the start symbol, A is a non-terminal and a and b are terminals.

- $S \rightarrow aS \mid A$
- $A \rightarrow aAb \mid bAa \mid \epsilon$

For the string " $aabbaab$ " how many steps are required to derive the string and how many parse trees are there?

- A. 6 and 1
- B. 6 and 2
- C. 7 and 2
- D. 4 and 2

gateit-2008 compiler-design parsing normal

Answer key

2.23

Register Allocation (6)

Practice Test: Test 1 (7Q)

2.23.1 Register Allocation: GATE CSE 1997 | Question: 4.9



The expression $(a * b) * c \text{ op } \dots$

where 'op' is one of '+', '*', and '^' (exponentiation) can be evaluated on a CPU with single register without storing the value of $(a * b)$ if

- A. 'op' is '+' or '*'
- B. 'op' is '^' or '*'
- C. 'op' is '^' or '+'
- D. not possible to evaluate without storing

gate1997 compiler-design register-allocation normal

Answer key

2.23.2 Register Allocation: GATE CSE 2004 | Question: 10



Consider the grammar rule $E \rightarrow E_1 - E_2$ for arithmetic expressions. The code generated is targeted to a CPU having a single user register. The subtraction operation requires the first operand to be in the register.

If $E1$ and $E2$ do not have any common sub expression, in order to get the shortest possible code

- A. $E1$ should be evaluated first
- B. $E2$ should be evaluated first
- C. Evaluation of $E1$ and $E2$ should necessarily be interleaved
- D. Order of evaluation of $E1$ and $E2$ is of no consequence

gatecse-2004 compiler-design register-allocation normal

Answer key

2.23.3 Register Allocation: GATE CSE 2010 | Question: 37



The program below uses six temporary variables a, b, c, d, e, f .

```
a = 1
b = 10
c = 20
d = a + b
e = c + d
f = c + e
b = c + e
e = b + f
d = 5 + e
return d + f
```

Assuming that all operations take their operands from registers, what is the minimum number of registers needed to execute this program without spilling?

- A. 2
- B. 3
- C. 4
- D. 6

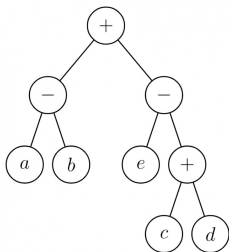
gatecse-2010 compiler-design register-allocation normal

Answer key

2.23.4 Register Allocation: GATE CSE 2011 | Question: 36



Consider evaluating the following expression tree on a machine with load-store architecture in which memory can be accessed only through load and store instructions. The variables a, b, c, d , and e are initially stored in memory. The binary operators used in this expression tree can be evaluated by the machine only when operands are in registers. The instructions produce result only in a register. If no intermediate results can be stored in memory, what is the minimum number of registers needed to evaluate this expression?



- A. 2
- B. 9
- C. 5
- D. 3

gatecse-2011 compiler-design register-allocation normal

Answer key

2.23.5 Register Allocation: GATE CSE 2013 | Question: 48



The following code segment is executed on a processor which allows only register operands in its instructions. Each instruction can have atmost two source operands and one destination operand. Assume that all variables are dead after this code segment.

```
c = a + b;
d = c * a;
e = c + a;
x = c * c;
if (x > a) {
    y = a * a;
```



```

}
else {
d = d * d;
e = e * e;
}

```

Q.48 Suppose the instruction set architecture of the processor has only two registers. The only allowed compiler optimization is code motion, which moves statements from one place to another while preserving correctness. What is the minimum number of spills to memory in the compiled code?

- A. 0 B. 1 C. 2 D. 3

gatecse-2013 normal compiler-design register-allocation

Answer key

2.23.6 Register Allocation: GATE CSE 2017 | Set 1 | Question: 52



Consider the expression $(a - 1) * (((b + c)/3) + d)$. Let X be the minimum number of registers required by an *optimal* code generation (without any register spill) algorithm for a load/store architecture, in which

- A. only load and store instructions can have memory operands and
B. arithmetic instructions can have only register or immediate operands.

The value of X is _____ .

gatecse-2017-set1 compiler-design register-allocation normal numerical-answers

Answer key

2.24 Runtime Environment (22)

Practice Tests: [Test 1 \(15Q\)](#) [Test 2 \(2Q\)](#)

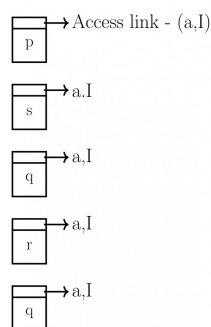
2.24.1 Runtime Environment: GATE CSE 1988 | Question: 2xii



Consider the following program skeleton and below figure which shows activation records of procedures involved in the calling sequence.

$$p \rightarrow s \rightarrow q \rightarrow r \rightarrow q.$$

Write the access links of the activation records to enable correct access and variables in the procedures from other procedures involved in the calling sequence



```

procedure p;
  procedure q;
    procedure r;
      begin
        q
      end r;
    begin
      r
    end q;
  procedure s;
    begin
      q
    end s;
  begin
    s

```

end p;

gate1988 normal descriptive runtime-environment compiler-design

Answer key

2.24.2 Runtime Environment: GATE CSE 1989 | Question: 10a



Will recursion work correctly in a language with static allocation of all variables? Explain.

gate1989 descriptive compiler-design runtime-environment

Answer key

2.24.3 Runtime Environment: GATE CSE 1989 | Question: 8b



Indicate the result of the following program if the language uses (i) static scope rules and (ii) dynamic scope rules.

```
var x, y:integer;
procedure A (var z:integer);
var x:integer;
begin x:=1; B; z:= x end;
procedure B;
begin x:=x+1 end;
begin
x:=5; A(y); write (y)
...end.
```

gate1989 descriptive compiler-design runtime-environment

Answer key

2.24.4 Runtime Environment: GATE CSE 1990 | Question: 2-v



Match the pairs in the following questions:

(a) Pointer data type	(p) Type conversion
(b) Activation record	(q) Dynamic data structure
(c) Repeat-until	(r) Recursion
(d) Coercion	(s) Nondeterministic loop

gate1990 match-the-following compiler-design runtime-environment recursion

Answer key

2.24.5 Runtime Environment: GATE CSE 1990 | Question: 4-v



State whether the following statements are TRUE or FALSE with reason:

The Link-load-and-go loading scheme required less storage space than the link-and-go loading scheme.

gate1990 true-false compiler-design runtime-environment

Answer key

2.24.6 Runtime Environment: GATE CSE 1993 | Question: 7.7



A part of the system software which under all circumstances must reside in the main memory is:

- A. text editor B. assembler C. linker D. loader
- E. none of the above

gate1993 compiler-design runtime-environment easy

Answer key

2.24.7 Runtime Environment: GATE CSE 1995 | Question: 1.14



A linker is given object modules for a set of programs that were compiled separately. What information need not be included in an object module?

- A. Object code
- B. Relocation bits
- C. Names and locations of all external symbols defined in the object module
- D. Absolute addresses of internal symbols

gate1995 compiler-design runtime-environment normal

Answer key

2.24.8 Runtime Environment: GATE CSE 1996 | Question: 2.17



The correct matching for the following pairs is

(A) Activation record	(1) Linking loader
(B) Location counter	(2) Garbage collection
(C) Reference counts	(3) Subroutine call
(D) Address relocation	(4) Assembler

- A. A-3 B-4 C-1 D-2 B. A-4 B-3 C-1 D-2 C. A-4 B-3 C-2 D-1 D. A-3 B-4 C-2 D-1

gate1996 compiler-design easy runtime-environment

Answer key

2.24.9 Runtime Environment: GATE CSE 1997 | Question: 1.10



Heap allocation is required for languages.

- A. that support recursion
- B. that support dynamic data structure
- C. that use dynamic scope rules
- D. None of the above

gate1997 compiler-design easy runtime-environment

Answer key

2.24.10 Runtime Environment: GATE CSE 1997 | Question: 1.8



A language L allows declaration of arrays whose sizes are not known during compilation. It is required to make efficient use of memory. Which one of the following is true?

- A. A compiler using static memory allocation can be written for L
- B. A compiler cannot be written for L ; an interpreter must be used
- C. A compiler using dynamic memory allocation can be written for L
- D. None of the above

gate1997 compiler-design easy runtime-environment

Answer key

2.24.11 Runtime Environment: GATE CSE 1998 | Question: 1.25, ISRO2008-41



In a resident – OS computer, which of the following systems must reside in the main memory under all situations?

- A. Assembler B. Linker C. Loader D. Compiler

gate1998 compiler-design runtime-environment normal isro2008

Answer key

2.24.12 Runtime Environment: GATE CSE 1998 | Question: 1.28



A linker reads four modules whose lengths are 200, 800, 600 and 500 words, respectively. If they are loaded in that order, what are the relocation constants?

- A. 0, 200, 500, 600
- B. 0, 200, 1000, 1600
- C. 200, 500, 600, 800
- D. 200, 700, 1300, 2100

gate1998 compiler-design runtime-environment normal

Answer key

2.24.13 Runtime Environment: GATE CSE 1998 | Question: 2.15



Faster access to non-local variables is achieved using an array of pointers to activation records called a

- A. stack
- B. heap
- C. display
- D. activation tree

gate1998 programming compiler-design normal runtime-environment

Answer key

2.24.14 Runtime Environment: GATE CSE 2001 | Question: 1.17



The process of assigning load addresses to the various parts of the program and adjusting the code and the data in the program to reflect the assigned addresses is called

- A. Assembly
- B. parsing
- C. Relocation
- D. Symbol resolution

gatecse-2001 compiler-design runtime-environment easy

Answer key

2.24.15 Runtime Environment: GATE CSE 2002 | Question: 2.20



Dynamic linking can cause security concerns because

- A. Security is dynamic
- B. The path for searching dynamic libraries is not known till runtime
- C. Linking is insecure
- D. Cryptographic procedures are not available for dynamic linking

gatecse-2002 compiler-design runtime-environment easy

Answer key

2.24.16 Runtime Environment: GATE CSE 2008 | Question: 54



Which of the following are true?

- I. A programming language which does not permit global variables of any kind and has no nesting of procedures/functions, but permits recursion can be implemented with static storage allocation
- II. Multi-level access link (or display) arrangement is needed to arrange activation records only if the programming language being implemented has nesting of procedures/functions
- III. Recursion in programming languages cannot be implemented with dynamic storage allocation
- IV. Nesting procedures/functions and recursion require a dynamic heap allocation scheme and cannot be implemented with a stack-based allocation scheme for activation records
- V. Programming languages which permit a function to return a function as its result cannot be implemented with a stack-based storage allocation scheme for activation records

- A. II and V only
- B. I, III and IV only
- C. I, II and V only
- D. II, III and V only

gatecse-2008 compiler-design difficult runtime-environment

Answer key

2.24.17 Runtime Environment: GATE CSE 2010 | Question: 14



Which languages necessarily need heap allocation in the runtime environment?

- A. Those that support recursion.
- B. Those that use dynamic scoping.
- C. Those that allow dynamic data structure.
- D. Those that use global variables.

gatecse-2010 compiler-design easy runtime-environment

Answer key

2.24.18 Runtime Environment: GATE CSE 2012 | Question: 36



Consider the program given below, in a block-structured pseudo-language with lexical scoping and nesting of procedures permitted.

```

Program main;
Var ...

Procedure A1;
Var ...
Call A2;
End A1

Procedure A2;
Var ...

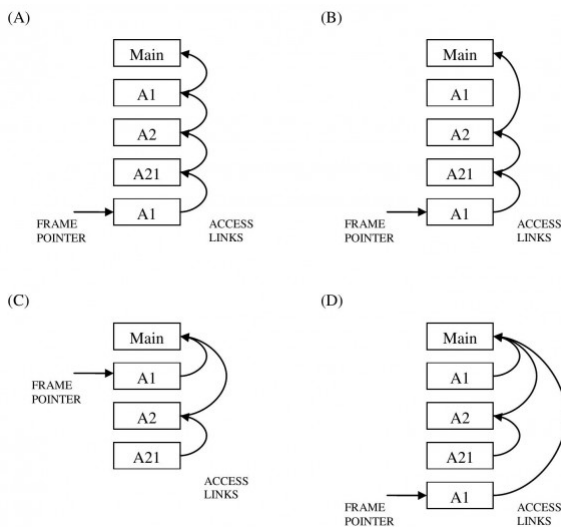
  Procedure A21;
  Var ...
  Call A1;
  End A21

  Call A21;
End A2

Call A1;
End main.
    
```

Consider the calling chain: $\text{Main} \rightarrow \text{A1} \rightarrow \text{A2} \rightarrow \text{A21} \rightarrow \text{A1}$

The correct set of activation records along with their access links is given by:



gatecse-2012 compiler-design runtime-environment normal

Answer key

2.24.19 Runtime Environment: GATE CSE 2014 | Set 2 | Question: 18



Which one of the following is **NOT** performed during compilation?

- A. Dynamic memory allocation
- B. Type checking
- C. Symbol table management
- D. Inline expansion

gatecse-2014-set2 compiler-design easy runtime-environment

2.24.20 Runtime Environment: GATE CSE 2014 | Set 3 | Question: 18



Which of the following statements are CORRECT?

1. Static allocation of all data areas by a compiler makes it impossible to implement recursion.
2. Automatic garbage collection is essential to implement recursion.
3. Dynamic allocation of activation records is essential to implement recursion.
4. Both heap and stack are essential to implement recursion.

A. 1 and 2 only B. 2 and 3 only C. 3 and 4 only D. 1 and 3 only

gatecse-2014-set3 compiler-design runtime-environment normal

2.24.21 Runtime Environment: GATE CSE 2021 | Set 1 | Question: 4



Consider the following statements.

- S_1 : The sequence of procedure calls corresponds to a preorder traversal of the activation tree.
- S_2 : The sequence of procedure returns corresponds to a postorder traversal of the activation tree.

Which one of the following options is correct?

A. S_1 is true and S_2 is false B. S_1 is false and S_2 is true
C. S_1 is true and S_2 is true D. S_1 is false and S_2 is false

gatecse-2021-set1 runtime-environment normal one-mark

2.24.22 Runtime Environment: GATE CSE 2023 | Question: 26



Consider the following program:

```

int main()
{
    f1 ();
    f2(2);
    f3();
    return (0);
}

int f1 ()
{
    return(1);
}

int f2 (int X)
{
    f3();
    if (X==1);
        return f1 ();
    else
        return (X * f2 (X - 1));
}

int f3 ()
{
    return (5);
}

```

Which one of the following options represents the activation tree corresponding to the main function?



gatecse-2023 compiler-design runtime-environment two-marks

2.25

Static Single Assignment (3)

Practice Test: Test 1 (6Q)

2.25.1 Static Single Assignment: GATE CSE 2015 | Set 1 | Question: 55



The least number of temporary variables required to create a three-address code in static single assignment form for the expression $q + r/3 + s - t * 5 + u * v/w$ is _____.

gatescse-2015-set1 compiler-design intermediate-code normal numerical-answers static-single-assignment

Answer key

2.25.2 Static Single Assignment: GATE CSE 2016 | Set 1 | Question: 19



Consider the following code segment.

```
x = u - t;
y = x * v;
x = y + w;
y = t - z;
y = x * y;
```

The minimum number of *total* variables required to convert the above code segment to *static single assignment* form is _____.

gatescse-2016-set1 compiler-design static-single-assignment normal numerical-answers

Answer key

2.25.3 Static Single Assignment: GATE CSE 2017 | Set 1 | Question: 12



Consider the following intermediate program in three address code

```
p = a - b
q = p * c
p = u * v
q = p + q
```

Which one of the following corresponds to a *static single assignment* form of the above code?

A.	p1 = a - b q1 = p1 * c p1 = u * v q1 = p1 + q1	B.	p3 = a - b q4 = p3 * c p4 = u * v q5 = p4 + q4	C.	p1 = a - b q1 = p2 * c p3 = u * v q2 = p4 + q3	D.	p1 = a - b q1 = p * c p2 = u * v q2 = p + q
----	---	----	---	----	---	----	--

gatescse-2017-set1 compiler-design intermediate-code normal static-single-assignment

Answer key

2.26

Symbol Table (1)

2.26.1 Symbol Table: GATE CSE 2025 | Set 1 | Question: 2



Which ONE of the following statements is **FALSE** regarding the symbol table?

- A. Symbol table is responsible for keeping track of the scope of variables.
- B. Symbol table can be implemented using a binary search tree.
- C. Symbol table is not required after the parsing phase.
- D. Symbol table is created during the lexical analysis phase.

gatescse2025-set1 compiler-design symbol-table easy one-mark

Answer key

2.27

Syntax Directed Translation (19)

Practice Tests: [Test 1 \(15Q\)](#) [Test 2 \(10Q\)](#)

2.27.1 Syntax Directed Translation: GATE CSE 1992 | Question: 11a



Write syntax directed definitions (semantic rules) for the following grammar to add the type of each identifier to its entry in the symbol table during semantic analysis. Rewriting the grammar is not permitted and semantic rules are to be added to the ends of productions only.

- $D \rightarrow TL;$
- $T \rightarrow \text{int}$
- $T \rightarrow \text{real}$
- $L \rightarrow L, id$
- $L \rightarrow id$

gate1992 compiler-design syntax-directed-translation normal descriptive

Answer key

2.27.2 Syntax Directed Translation: GATE CSE 1995 | Question: 2.10



A shift reduce parser carries out the actions specified within braces immediately after reducing with the corresponding rule of grammar

- $S \rightarrow xxW \{\text{print "1"}\}$
- $S \rightarrow y \{\text{print "2"}\}$
- $W \rightarrow Sz \{\text{print "3"}\}$

What is the translation of $xxxxyz$ using the syntax directed translation scheme described by the above rules?

- A. 23131 B. 11233 C. 11231 D. 33211

gate1995 compiler-design grammar syntax-directed-translation normal

Answer key

2.27.3 Syntax Directed Translation: GATE CSE 1996 | Question: 20



Consider the syntax-directed translation schema (SDTS) shown below:

- $E \rightarrow E + E \{\text{print "+"}\}$
- $E \rightarrow E * E \{\text{print "."}\}$
- $E \rightarrow id \{\text{print id.name}\}$
- $E \rightarrow (E)$

An LR-parser executes the actions associated with the productions immediately after a reduction by the corresponding production. Draw the parse tree and write the translation for the sentence.

$(a + b) * (c + d)$, using SDTS given above.

gate1996 compiler-design syntax-directed-translation normal descriptive

Answer key

2.27.4 Syntax Directed Translation: GATE CSE 1998 | Question: 23



Let the attribute ' val ' give the value of a binary number generated by S in the following grammar:

- $S \rightarrow L.L \mid L$
- $L \rightarrow LB \mid B$
- $B \rightarrow 0 \mid 1$

For example, an input 101.101 gives $S.val = 5.625$

Construct a syntax directed translation scheme using only synthesized attributes, to determine $S.val$.

gate1998 compiler-design syntax-directed-translation normal descriptive

Answer key

2.27.5 Syntax Directed Translation: GATE CSE 2000 | Question: 19



Consider the syntax directed translation scheme (SDTS) given in the following. Assume attribute evaluation with bottom-up parsing, i.e., attributes are evaluated immediately after a reduction.

- $E \rightarrow E_1 * T \quad \{E.val = E_1.val * T.val\}$
- $E \rightarrow T \quad \{E.val = T.val\}$
- $T \rightarrow F - T_1 \quad \{T.val = F.val - T_1.val\}$
- $T \rightarrow F \quad \{T.val = F.val\}$
- $F \rightarrow 2 \quad \{F.val = 2\}$
- $F \rightarrow 4 \quad \{F.val = 4\}$

- A. Using this SDTS, construct a parse tree for the expression $4 - 2 - 4 * 2$ and also compute its $E.val$.
- B. It is required to compute the total number of reductions performed to parse a given input. Using synthesized attributes only, modify the SDTS given, without changing the grammar, to find $E.red$, the number of reductions performed while reducing an input to E .

gatecse-2000 compiler-design syntax-directed-translation normal descriptive

Answer key

2.27.6 Syntax Directed Translation: GATE CSE 2001 | Question: 17



The syntax of the repeat-until statement is given by the following grammar

$S \rightarrow \text{repeat } S_1 \text{ until } E$

where E stands for expressions, S and S_1 stand for statements. The non-terminals S and S_1 have an attribute code that represents generated code. The non-terminal E has two attributes. The attribute code represents generated code to evaluate the expression and store its value in a distinct variable, and the attribute varName contains the name of the variable in which the truth value is stored. The truth-value stored in the variable is 1 if E is true, 0 if E is false.

Give a syntax-directed definition to generate three-address code for the repeat-until statement. Assume that you can call a function newlabel() that returns a distinct label for a statement. Use the operator '\n' to concatenate two strings and the function gen(s) to generate a line containing the string s.

gatecse-2001 compiler-design syntax-directed-translation normal descriptive

Answer key

2.27.7 Syntax Directed Translation: GATE CSE 2003 | Question: 18



In a bottom-up evaluation of a syntax directed definition, inherited attributes can

- A. always be evaluated
- B. be evaluated only if the definition is L-attributed
- C. be evaluated only if the definition has synthesized attributes
- D. never be evaluated

gatecse-2003 compiler-design syntax-directed-translation normal

Answer key

2.27.8 Syntax Directed Translation: GATE CSE 2003 | Question: 59



Consider the syntax directed definition shown below.

$$\begin{array}{ll}
 S \rightarrow \text{id} := E & \{ \text{gen}(\text{id.place} = E.\text{place};); \} \\
 E \rightarrow E_1 + E_2 & \{ t = \text{newtemp}(); \\
 & \text{gen}(t = E_1.\text{place} + E_2.\text{place};); \\
 & E.\text{place} = t; \} \\
 E \rightarrow \text{id} & \{ E.\text{place} = \text{id.place}; \}
 \end{array}$$

Here, *gen* is a function that generates the output code, and *newtemp* is a function that returns the name of a new temporary variable on every call. Assume that t'_i s are the temporary variable names generated by *newtemp*. For the statement ' $X := Y + Z$ ', the 3-address code sequence generated by this definition is

- A. $X = Y + Z$
 C. $t_1 = Y; t_2 = t_1 + Z; X = t_2$
 B. $t_1 = Y + Z; X = t_1$
 D. $t_1 = Y; t_2 = Z; t_3 = t_1 + t_2; X = t_3$

gatecse-2003 compiler-design syntax-directed-translation normal

Answer key

2.27.9 Syntax Directed Translation: GATE CSE 2004 | Question: 45



Consider the grammar with the following translation rules and E as the start symbol

$$\begin{aligned} E &\rightarrow E_1 \# T & \{E.value = E_1.value * T.value\} \\ &| T & \{E.value = T.value\} \\ T &\rightarrow T_1 \& F & \{T.value = T_1.value + F.value\} \\ &| F & \{T.value = F.value\} \\ F &\rightarrow \text{num} & \{F.value = \text{num.value}\} \end{aligned}$$

Compute $E.value$ for the root of the parse tree for the expression $2 \# 3 \& 5 \# 6 \& 4$

- A. 200
 B. 180
 C. 160
 D. 40

gatecse-2004 compiler-design grammar normal syntax-directed-translation

Answer key

2.27.10 Syntax Directed Translation: GATE CSE 2016 | Set 1 | Question: 46



Consider the following Syntax Directed Translation Scheme ($SDTS$), with non-terminals $\{S, A\}$ and terminals $\{a, b\}$.

$$\begin{aligned} S &\rightarrow aA \quad \{\text{print } 1\} \\ S &\rightarrow a \quad \{\text{print } 2\} \\ A &\rightarrow Sb \quad \{\text{print } 3\} \end{aligned}$$

Using the above $SDTS$, the output printed by a bottom-up parser, for the input aab is:

- A. 1 3 2
 C. 2 3 1
 B. 2 2 3
 D. syntax error

gatecse-2016-set1 compiler-design syntax-directed-translation normal

Answer key

2.27.11 Syntax Directed Translation: GATE CSE 2019 | Question: 36



Consider the following grammar and the semantic actions to support the inherited type declaration attributes. Let X_1, X_2, X_3, X_4, X_5 , and X_6 be the placeholders for the non-terminals D, T, L or L_1 in the following table:

Production rule	Semantic action
$D \rightarrow TL$	$X_1.type = X_2.type$
$T \rightarrow \text{int}$	$T.type = \text{int}$
$T \rightarrow \text{float}$	$T.type = \text{float}$
$L \rightarrow L_1, id$	$X_3.type = X_4.type$ $\text{addType}(id.entry, X_5.type)$
$L \rightarrow id$	$\text{addType}(id.entry, X_6.type)$

Which one of the following are appropriate choices for X_1, X_2, X_3 and X_4 ?

- A. $X_1 = L, X_2 = T, X_3 = L_1, X_4 = L$

- B. $X_1 = T, X_2 = L, X_3 = L_1, X_4 = T$
 C. $X_1 = L, X_2 = L, X_3 = L_1, X_4 = T$
 D. $X_1 = T, X_2 = L, X_3 = T, X_4 = L_1$

gatecse-2019 compiler-design syntax-directed-translation two-marks

Answer key

2.27.12 Syntax Directed Translation: GATE CSE 2020 | Question: 33



Consider the productions $A \rightarrow PQ$ and $A \rightarrow XY$. Each of the five non-terminals A, P, Q, X , and Y has two attributes: s is a synthesized attribute, and i is an inherited attribute. Consider the following rules.

- Rule 1 : $P.i = A.i + 2$, $Q.i = P.i + A.i$, and $A.s = P.s + Q.s$
- Rule 2 : $X.i = A.i + Y.s$ and $Y.i = X.s + A.i$

Which one of the following is TRUE?

- A. Both Rule 1 and Rule 2 are L -attributed.
 B. Only Rule 1 is L -attributed.
 C. Only Rule 2 is L -attributed.
 D. Neither Rule 1 nor Rule 2 is L -attributed.

gatecse-2020 compiler-design syntax-directed-translation two-marks

Answer key

2.27.13 Syntax Directed Translation: GATE CSE 2021 | Set 1 | Question: 26



Consider the following grammar (that admits a series of declarations, followed by expressions) and the associated syntax directed translation (SDT) actions, given as pseudo-code

$P \rightarrow D^*E^*$
 $D \rightarrow \text{int ID}\{\text{record that ID.lexeme is of type int}\}$
 $D \rightarrow \text{bool ID}\{\text{record that ID.lexeme is of type bool}\}$
 $E \rightarrow E_1 + E_2\{\text{check that } E_1.\text{type} = E_2.\text{type} = \text{int}; \text{set } E.\text{type} := \text{int}\}$
 $E \rightarrow !E_1\{\text{check that } E_1.\text{type} = \text{bool}; \text{set } E.\text{type} := \text{bool}\}$
 $E \rightarrow \text{ID}\{\text{set } E.\text{type} := \text{int}\}$

With respect to the above grammar, which one of the following choices is correct?

- A. The actions can be used to correctly type-check any syntactically correct program
 B. The actions can be used to type-check syntactically correct integer variable declarations and integer expressions
 C. The actions can be used to type-check syntactically correct boolean variable declarations and boolean expressions.
 D. The actions will lead to an infinite loop

gatecse-2021-set1 compiler-design syntax-directed-translation two-marks

Answer key

2.27.14 Syntax Directed Translation: GATE CSE 2022 | Question: 55



Consider the following grammar along with translation rules.

$S \rightarrow S_1 \# T \quad \{S_{\text{val}} = S_1.\text{val} * T.\text{val}\}$
 $S \rightarrow T \quad \{S_{\text{val}} = T.\text{val}\}$
 $T \rightarrow T_1 \% R \quad \{T_{\text{val}} = T_1.\text{val} \div R.\text{val}\}$
 $T \rightarrow R \quad \{T_{\text{val}} = R.\text{val}\}$
 $R \rightarrow \text{id} \quad \{R_{\text{val}} = \text{id.val}\}$

Here $\#$ and $\%$ are operators and id is a token that represents an integer and $id.val$ represents the corresponding integer value. The set of non-terminals is $\{S, T, R, P\}$ and a subscripted non-terminal indicates an instance of the non-terminal.

Using this translation scheme, the computed value of $S.val$ for root of the parse tree for the expression $20\#10\%5\#8\%2\%2$ is _____.

gatecse-2022 numerical-answers compiler-design syntax-directed-translation two-marks

Answer key

2.27.15 Syntax Directed Translation: GATE CSE 2023 | Question: 50



Consider the syntax directed translation given by the following grammar and semantic rules. Here N, I, F and B are non-terminals. N is the starting non-terminal, and $\#, 0$ and 1 are lexical tokens corresponding to input letters “ $\#$ ”, “ 0 ” and “ 1 ”, respectively. $X.val$ denotes the synthesized attribute (a numeric value) associated with a non-terminal X . I_1 and F_1 denote occurrences of I and F on the right hand side of a production, respectively. For the tokens 0 and 1 , $0.val = 0$ and $1.val = 1$.

$$\begin{aligned} N &\rightarrow I\#F & N.val &= I.val + F.val \\ I &\rightarrow I_1B & I.val &= (2I_1.val) + B.val \\ I &\rightarrow B & I.val &= B.val \\ F &\rightarrow BF_1 & F.val &= \frac{1}{2}(B.val + F_1.val) \\ F &\rightarrow B & F.val &= \frac{1}{2}B.val \\ B &\rightarrow 0 & B.val &= 0.val \\ B &\rightarrow 1 & B.val &= 1.val \end{aligned}$$

The value computed by the translation scheme for the input string

10#011

is _____. (Rounded off to three decimal places)

gatecse-2023 compiler-design syntax-directed-translation numerical-answers two-marks

Answer key

2.27.16 Syntax Directed Translation: GATE CSE 2024 | Set 1 | Question: 27



Consider the following syntax-directed definition (SDD).

$S \rightarrow DHTU$	$\{S.val = D.val + H.val + T.val + U.val;\}$
$D \rightarrow "M" D_1$	$\{D.val = 5 + D_1.val;\}$
$D \rightarrow \epsilon$	$\{D.val = -5;\}$
$H \rightarrow "L" H_1$	$\{H.val = 5 * 10 + H_1.val;\}$
$H \rightarrow \epsilon$	$\{H.val = -10;\}$
$T \rightarrow "C" T_1$	$\{T.val = 5 * 100 + T_1.val;\}$
$T \rightarrow \epsilon$	$\{T.val = -5;\}$
$U \rightarrow "K"$	$\{U.val = 5;\}$

Given “MMLK” as the input, which one of the following options is the CORRECT value computed by the SDD (in the attribute $S.val$)?

- A. 45 B. 50 C. 55 D. 65

gatecse-2024-set1 compiler-design syntax-directed-translation two-marks

2.27.17 Syntax Directed Translation: GATE CSE 2024 | Set 2 | Question: 19



Which of the following statements is/are FALSE?

- A. An attribute grammar is a syntax-directed definition (SDD) in which the functions in the semantic rules have no side effects
- B. The attributes in a L -attributed definition cannot always be evaluated in a depth-first order
- C. Synthesized attributes can be evaluated by a bottom-up parser as the input is parsed
- D. All L -attributed definitions based on $LR(1)$ grammar can be evaluated using a bottom-up parsing strategy

gatecse-2024-set2 compiler-design syntax-directed-translation multiple-selects one-mark

2.27.18 Syntax Directed Translation: GATE CSE 2025 | Set 2 | Question: 12



Given the following syntax directed translation rules:

- Rule 1: $R \rightarrow AB\{B.i = R.i - 1; A.i = B.i; R.i = A.i + 1;\}$
- Rule 2: $P \rightarrow CD\{P.i = C.i + D.i; D.i = C.i + 2;\}$
- Rule 3: $Q \rightarrow EF\{Q.i = E.i + F.i;\}$

Which ONE is the CORRECT option among the following?

- A. Rule 1 is S -attributed and L -attributed; Rule 2 is S -attributed and not L -attributed; Rule 3 is neither S -attributed nor L -attributed.
- B. Rule 1 is neither S -attributed nor L -attributed; Rule 2 is S -attributed and L -attributed; Rule 3 is S -attributed and L -attributed.
- C. Rule 1 is neither S -attributed nor L -attributed; Rule 2 is not S -attributed and is L -attributed; Rule 3 is S -attributed and L -attributed.
- D. Rule 1 is S -attributed and not L -attributed; Rule 2 is not S -attributed and is L -attributed; Rule 3 is S -attributed and L -attributed.

gatecse2025-set2 compiler-design syntax-directed-translation one-mark

2.27.19 Syntax Directed Translation: GATE CSE 2026 | Set 1 | Question: 43



Consider the following two syntax-directed definitions SDD1 and SDD2 for type declarations.

SDD1	
Grammar (G1)	Semantic Rules
$D \rightarrow T V$	$D.type = T.type$ $V.type = T.type$
$T \rightarrow \text{int}$	$T.type = \text{int}$
$T \rightarrow \text{float}$	$T.type = \text{float}$
$V \rightarrow V_1 \text{ id}$	$V_1.type = V.type$ $\text{put}(\text{id.entry}, V.type)$
$V \rightarrow \text{id}$	$\text{put}(\text{id.entry}, V.type)$

SDD2	
Grammar (G2)	Semantic Rules
$D \rightarrow D_1 \text{ id}$	$D.type = D_1.type$ $\text{put}(\text{id.entry}, D_1.type)$
$D \rightarrow T \text{ id}$	$D.type = T.type$ $\text{put}(\text{id.entry}, T.type)$

$T \rightarrow \text{int}$	$T.type = \text{int}$
$T \rightarrow \text{float}$	$T.type = \text{float}$

D is the start symbol, and int, float and id are the three terminals. The non-terminal V_1 is the same as V and the non-terminal D_1 is the same as D . Here, the subscript is used to differentiate the grammar symbols on the two sides of a production. The function put updates the symbol table with the type information for an identifier.

Let P and Q be the languages specified by grammars G_1 and G_2 , respectively.

Which of the following statements is/are true?

- A. The languages P and Q are the same
- B. SDD2 is S-attributed and contains only synthesized attributes
- C. SDD1 is L-attributed and contains only inherited attributes
- D. The specifications of SDD1 and SDD2 are such that the same entries get added to the symbol table

gatecse-2026-set1 compiler-design syntax-directed-translation two-marks multiple-selects

Answer key

2.28

Variable Scope (2)

2.28.1 Variable Scope: GATE CSE 1987 | Question: 1-xix



Study the following program written in a block-structured language:

```

Var x, y: interger;
procedure P(n: interger);
begin
  x := (n+2)/(n-3);
end;

procedure Q
Var x, y: interger;
begin
  x := 3;
  y := 4;
  P(y);
  Write(x)           __ (1)
end;

begin
  x := 7;
  y := 8;
  Q;
  Write(x);           __ (2)
end.

```

What will be printed by the write statements marked (1) and (2) in the program if the variables are statically scoped?

- A. 3,6
- B. 6,7
- C. 3,7
- D. None of the above.

gate1987 compiler-design variable-scope runtime-environment

Answer key

2.28.2 Variable Scope: GATE CSE 1987 | Question: 1-xx



For the program given below what will be printed by the write statements marked (1) and (2) in the program if the variables are dynamically scoped?

```

Var x, y: interger;
procedure P(n: interger);
begin
  x := (n+2)/(n-3);
end;

procedure Q
Var x, y: interger;
begin
  x := 3;
  y := 4;

```

```

P(y);
Write(x);    ___(1)
end;

begin
x:=7;
y:=8;
Q;
Write(x);    ___(2)
end.

```

- A. 3,6 B. 6,7 C. 3,7 D. None of the above

gate1987 compiler-design variable-scope runtime-environment

Answer key

2.29 Viable Prefix (1)

2.29.1 Viable Prefix: GATE CSE 2015 | Set 1 | Question: 13



Which one of the following is TRUE at any valid state in shift-reduce parsing?

- A. Viable prefixes appear only at the bottom of the stack and not inside
B. Viable prefixes appear only at the top of the stack and not inside
C. The stack contains only a set of viable prefixes
D. The stack never contains viable prefixes

gatecse-2015-set1 compiler-design parsing normal viable-prefix lr-parser

Answer key

Answer Keys

2.1.1	C	2.2.1	B	2.2.2	B;C	2.3.1	N/A	2.3.2	TBA
2.3.3	N/A	2.3.4	A;B;C	2.3.5	True	2.3.6	N/A	2.3.7	True
2.3.8	False	2.3.9	B	2.4.1	C	2.5.1	6:6	2.5.2	A
2.6.1	A	2.6.2	D	2.6.3	A	2.6.4	D	2.6.5	7
2.6.6	D	2.6.7	B	2.6.8	D	2.7.1	C	2.7.2	N/A
2.7.3	A	2.7.4	B	2.7.5	C	2.7.6	B	2.7.7	C
2.7.8	B	2.7.9	D	2.7.10	C	2.7.11	B	2.7.12	B
2.7.13	C	2.8.1	13:13	2.9.1	A	2.9.2	6 : 6	2.10.1	A
2.10.2	6	2.11.1	A;B	2.11.2	A	2.11.3	C	2.11.4	31
2.11.5	A	2.11.6	A;D	2.12.1	N/A	2.12.2	N/A	2.12.3	N/A
2.12.4	N/A	2.12.5	N/A	2.12.6	C	2.12.7	C	2.12.8	N/A
2.12.9	N/A	2.12.10	X	2.12.11	N/A	2.12.12	N/A	2.12.13	D
2.12.14	N/A	2.12.15	N/A	2.12.16	N/A	2.12.17	C	2.12.18	A
2.12.19	N/A	2.12.20	D	2.12.21	A	2.12.22	B	2.12.23	B
2.12.24	C	2.12.25	A	2.12.26	D	2.12.27	B	2.12.28	D
2.12.29	D	2.12.30	A	2.12.31	C	2.12.32	C	2.12.33	C
2.12.34	B	2.12.35	C	2.12.36	C	2.12.37	B	2.12.38	A
2.12.39	C	2.12.40	5	2.12.41	A	2.12.42	179	2.12.43	B;C
2.12.44	C;D	2.12.45	A	2.12.46	C	2.12.47	D	2.13.1	N/A
2.13.2	N/A	2.13.3	N/A	2.13.4	C	2.13.5	A	2.13.6	C

2.13.7	A
2.14.1	B;C;D
2.14.6	B
2.14.11	A
2.14.16	5
2.15.1	A
2.15.6	D
2.17.2	A
2.19.2	D
2.20.3	B
2.20.8	A
2.21.4	N/A
2.21.9	A
2.21.14	D
2.22.5	C
2.22.10	D
2.22.15	D
2.22.20	A;D
2.23.3	B
2.24.2	N/A
2.24.7	C
2.24.12	B
2.24.17	C
2.24.22	A
2.27.1	N/A
2.27.6	N/A
2.27.11	A
2.27.16	A
2.28.2	B

2.13.8	B
2.14.2	C
2.14.7	D
2.14.12	D
2.14.17	D
2.15.2	C
2.16.1	B
2.17.3	D
2.19.3	A
2.20.4	N/A
2.20.9	6
2.21.5	A;D
2.21.10	N/A
2.22.1	A
2.22.6	N/A
2.22.11	C
2.22.16	B
2.22.21	A
2.23.4	D
2.24.3	N/A
2.24.8	D
2.24.13	C
2.24.18	D
2.25.1	8
2.27.2	A
2.27.7	B
2.27.12	B
2.27.17	B;D
2.29.1	C

2.13.9	D
2.14.3	B
2.14.8	B
2.14.13	7
2.14.18	9
2.15.3	B
2.16.2	C
2.18.1	A
2.19.4	D
2.20.5	B
2.21.1	10
2.21.6	N/A
2.21.11	X
2.22.2	N/A
2.22.7	B
2.22.12	B
2.22.17	C
2.22.22	A
2.23.5	B
2.24.4	N/A
2.24.9	B
2.24.14	C
2.24.19	A
2.25.2	10
2.27.3	N/A
2.27.8	B
2.27.13	B
2.27.18	C

2.13.10	D
2.14.4	B
2.14.9	D
2.14.14	C
2.14.19	C
2.15.4	C
2.16.3	C
2.18.2	C
2.20.1	N/A
2.20.6	B
2.21.2	N/A
2.21.7	N/A
2.21.12	D
2.22.3	N/A
2.22.8	A
2.22.13	A
2.22.18	D
2.23.1	A
2.23.6	2
2.24.5	True
2.24.10	C
2.24.15	B
2.24.20	D
2.25.3	B
2.27.4	N/A
2.27.9	C
2.27.14	80
2.27.19	A;B;D

2.13.11	A
2.14.5	D
2.14.10	C
2.14.15	8 : 8
2.14.20	D
2.15.5	A
2.17.1	C
2.19.1	N/A
2.20.2	A
2.20.7	9
2.21.3	A;D
2.21.8	N/A
2.21.13	D
2.22.4	N/A
2.22.9	N/A
2.22.14	A
2.22.19	1024
2.23.2	B
2.24.1	N/A
2.24.6	D
2.24.11	C
2.24.16	A
2.24.21	C
2.26.1	C
2.27.5	N/A
2.27.10	C
2.27.15	2.375
2.28.1	A